

```
01 // FILE: src\main\java\com\sqlteacher\application\ai\AiAvailability.java
02 package com.sqlteacher.application.ai;
03 public enum AiAvailability {
04     AVAILABLE,
05     UNAVAILABLE
06 }
07 // FILE: src\main\java\com\sqlteacher\application\ai\AiCompletionRequest.java
08 package com.sqlteacher.application.ai;
09 import java.time.Duration;
10 import java.util.Objects;
11 public record AiCompletionRequest(
12     String model,
13     String prompt,
14     Duration timeout
15 ) {
16     public AiCompletionRequest {
17         Objects.requireNonNull(model, "model must not be null");
18         Objects.requireNonNull(prompt, "prompt must not be null");
19         Objects.requireNonNull(timeout, "timeout must not be null");
20         if (model.isBlank()) {
21             throw new IllegalArgumentException("model must not be blank");
22         }
23         if (prompt.isBlank()) {
24             throw new IllegalArgumentException("prompt must not be blank");
25         }
26         if (timeout.isZero() || timeout.isNegative()) {
27             throw new IllegalArgumentException("timeout must be positive");
28         }
29     }
30 }
31 // FILE: src\main\java\com\sqlteacher\application\ai\AiCompletionResult.java
32 package com.sqlteacher.application.ai;
33 import java.util.Objects;
34 public final class AiCompletionResult {
35     private final boolean success;
36     private final String content;
37     private final String errorMessage;
38     private final String model;
39     private AiCompletionResult(boolean success, String content, String errorMessage, String model) {
40         this.success = success;
41         this.content = content;
42         this.errorMessage = errorMessage;
43         this.model = model;
44     }
45     public static AiCompletionResult success(String content, String model) {
46         Objects.requireNonNull(content, "content must not be null");
47         Objects.requireNonNull(model, "model must not be null");
48         return new AiCompletionResult(true, content, null, model);
49     }
50     public static AiCompletionResult failure(String errorMessage, String model) {
```

```
01         Objects.requireNonNull(errorMessage, "errorMessage must not be null");
02         Objects.requireNonNull(model, "model must not be null");
03         return new AiCompletionResult(false, null, errorMessage, model);
04     }
05     public boolean success() {
06         return success;
07     }
08     public String content() {
09         return content;
10     }
11     public String errorMessage() {
12         return errorMessage;
13     }
14     public String model() {
15         return model;
16     }
17 }
18 // FILE: src\main\java\com\sqlteacher\application\ai\AiModelProvider.java
19 package com.sqlteacher.application.ai;
20 @FunctionalInterface
21 public interface AiModelProvider {
22     AiCompletionResult complete(AiCompletionRequest request);
23     default String preferredModel() { return ""; }
24 }
25 // FILE: src\main\java\com\sqlteacher\application\ai\AiModelSelection.java
26 package com.sqlteacher.application.ai;
27 import java.util.List;
28 /** Snapshot of the locally installed AI models and the model selected for new requests. */
29 public record AiModelSelection(
30     List<String> installedModels,
31     String selectedModel,
32     String message
33 ) {
34     public AiModelSelection {
35         installedModels = installedModels == null ? List.of() : List.copyOf(installedModels);
36         selectedModel = selectedModel == null ? "" : selectedModel.strip();
37         message = message == null ? "" : message;
38         if (!selectedModel.isEmpty() && !installedModels.contains(selectedModel)) {
39             throw new IllegalArgumentException("selectedModel must be installed");
40         }
41     }
42     public boolean hasSelection() {
43         return !selectedModel.isEmpty();
44     }
45 }
46 // FILE: src\main\java\com\sqlteacher\application\ai\AiModelSelectionService.java
47 package com.sqlteacher.application.ai;
48 import java.util.List;
49 /** Discovers locally installed models and controls which model is used for future AI requests. */
50 public interface AiModelSelectionService {
```

```
01     AiModelSelection refresh();
02     AiModelSelection current();
03     AiModelSelection select(String model);
04     static AiModelSelectionService fixed(String model) {
05         AiModelSelection selection = new AiModelSelection(List.of(model), model, "Fixed model");
06         return new AiModelSelectionService() {
07             @Override
08             public AiModelSelection refresh() {
09                 return selection;
10             }
11             @Override
12             public AiModelSelection current() {
13                 return selection;
14             }
15             @Override
16             public AiModelSelection select(String selectedModel) {
17                 if (!model.equals(selectedModel)) {
18                     throw new IllegalArgumentException("model is not installed: " + selectedModel);
19                 }
20                 return selection;
21             }
22         };
23     }
24 }

25 // FILE: src\main\java\com\sqlteacher\application\ai\AiStatus.java
26 package com.sqlteacher.application.ai;
27 public record AiStatus(
28     AiAvailability status,
29     String provider,
30     String endpoint,
31     int modelCount,
32     String message
33 ) {
34     public boolean available() {
35         return status == AiAvailability.AVAILABLE;
36     }
37 }

38 // FILE: src\main\java\com\sqlteacher\application\ai\AiStatusService.java
39 package com.sqlteacher.application.ai;
40 public interface AiStatusService {
41     AiStatus checkStatus();
42 }

43 // FILE: src\main\java\com\sqlteacher\application\ai\NetworkAiSettingsService.java
44 package com.sqlteacher.application.ai;
45 import java.net.URI;
46 import java.util.Optional;
47 public interface NetworkAiSettingsService {
48     void configure(URI endpoint, String model, char[] apiKey);
49     Optional<OpenAiCompatibleConfiguration> current();
50     void clear();
```

```
01 }
02 // FILE: src\main\java\com\sqlteacher\application\ai\OpenAiCompatibleConfiguration.java
03 package com.sqlteacher.application.ai;
04 import java.net.URI;
05 import java.util.Objects;
06 import java.util.Arrays;
07 /**
08  * ████████ OpenAI-compatible ████████
09  * API Key ████████████████████████████████
10  */
11 public record OpenAiCompatibleConfiguration(URI endpoint, String model, char[] apiKey) {
12     public OpenAiCompatibleConfiguration {
13         Objects.requireNonNull(endpoint, "endpoint must not be null");
14         Objects.requireNonNull(model, "model must not be null");
15         Objects.requireNonNull(apiKey, "apiKey must not be null");
16         if (!"https".equalsIgnoreCase(endpoint.getScheme())) {
17             throw new IllegalArgumentException("Network AI endpoint must use HTTPS");
18         }
19         if (model.isBlank() || apiKey.length == 0) {
20             throw new IllegalArgumentException("model and apiKey must not be blank");
21         }
22         apiKey = apiKey.clone();
23     }
24     @Override
25     public char[] apiKey() {
26         return apiKey.clone();
27     }
28     public void destroy() { Arrays.fill(apiKey, '\0'); }
29 }
30 // FILE: src\main\java\com\sqlteacher\application\analytics\AnalyticsCsvExport.java
31 package com.sqlteacher.application.analytics;
32 import java.time.Instant;
33 public record AnalyticsCsvExport(String fileName, String utf8Content, Instant generatedAt) {
34     public AnalyticsCsvExport {
35         if (fileName == null || fileName.isBlank() || utf8Content == null || generatedAt == null) {
36             throw new IllegalArgumentException("CSV export values must not be null or blank");
37         }
38     }
39 }
40 // FILE: src\main\java\com\sqlteacher\application\analytics\AnalyticsFilter.java
41 package com.sqlteacher.application.analytics;
42 import java.time.Instant;
43 public record AnalyticsFilter(
44     Instant startInclusive,
45     Instant endExclusive,
46     String exerciseId,
47     String knowledgePoint,
48     String errorCode
49 ) {
50     public AnalyticsFilter {
```

```
01         if (startInclusive != null && endExclusive != null && !startInclusive.isBefore(endExclusive)) {
02             throw new IllegalArgumentException("startInclusive must be before endExclusive");
03         }
04         exerciseId = normalize(exerciseId);
05         knowledgePoint = normalize(knowledgePoint);
06         errorCode = normalize(errorCode);
07     }
08     public static AnalyticsFilter all() {
09         return new AnalyticsFilter(null, null, null, null, null);
10     }
11     private static String normalize(String value) {
12         return value == null || value.isBlank() ? null : value.trim();
13     }
14 }
15 // FILE: src\main\java\com\sqlteacher\application\analytics\AnalyticsOverview.java
16 package com.sqlteacher.application.analytics;
17 import java.time.Duration;
18 public record AnalyticsOverview(
19     int sessions,
20     int attempts,
21     int submissions,
22     int passedSubmissions,
23     double passRate,
24     double averageAttemptsPerCompletedExercise,
25     Duration averageSubmissionDuration,
26     int completedExercises,
27     int totalExercises,
28     double completionRate
29 ) {
30     public AnalyticsOverview {
31         if (sessions < 0 || attempts < 0 || submissions < 0 || passedSubmissions < 0
32             || completedExercises < 0 || totalExercises < 0) {
33             throw new IllegalArgumentException("analytics counts must not be negative");
34         }
35         if (passRate < 0 || passRate > 1 || completionRate < 0 || completionRate > 1
36             || averageAttemptsPerCompletedExercise < 0) {
37             throw new IllegalArgumentException("analytics rates must be within their valid range");
38         }
39         if (averageSubmissionDuration == null || averageSubmissionDuration.isNegative()) {
40             throw new IllegalArgumentException("averageSubmissionDuration must not be negative");
41         }
42     }
43 }
44 // FILE: src\main\java\com\sqlteacher\application\analytics\ErrorAnalytics.java
45 package com.sqlteacher.application.analytics;
46 public record ErrorAnalytics(String errorCode, int count) {
47     public ErrorAnalytics {
48         if (errorCode == null || errorCode.isBlank() || count < 1) {
49             throw new IllegalArgumentException("error analytics values are invalid");
50         }
51     }
52 }
```

```
01     }
02 }
03 // FILE: src\main\java\com\sqlteacher\application\analytics\ExerciseAnalyticsRow.java
04 package com.sqlteacher.application.analytics;
05 import java.time.Instant;
06 import java.util.Optional;
07 public record ExerciseAnalyticsRow(
08     String exerciseId,
09     String title,
10     String knowledgePoint,
11     int attempts,
12     int submissions,
13     int passedSubmissions,
14     int failedSubmissions,
15     double passRate,
16     boolean completed,
17     Instant lastAttemptAt
18 ) {
19     public ExerciseAnalyticsRow {
20         if (exerciseId == null || exerciseId.isBlank() || title == null || title.isBlank()
21             || knowledgePoint == null || knowledgePoint.isBlank()) {
22             throw new IllegalArgumentException("exercise analytics identity must not be blank");
23         }
24         if (attempts < 0 || submissions < 0 || passedSubmissions < 0 || failedSubmissions < 0
25             || passRate < 0 || passRate > 1) {
26             throw new IllegalArgumentException("exercise analytics values are invalid");
27         }
28     }
29     public Optional<Instant> lastAttempt() {
30         return Optional.ofNullable(lastAttemptAt);
31     }
32 }
33 // FILE: src\main\java\com\sqlteacher\application\analytics\KnowledgePointAnalytics.java
34 package com.sqlteacher.application.analytics;
35 public record KnowledgePointAnalytics(
36     String knowledgePoint,
37     int attempts,
38     int failedSubmissions,
39     int completedExercises,
40     int totalExercises,
41     double weaknessRate
42 ) {
43     public KnowledgePointAnalytics {
44         if (knowledgePoint == null || knowledgePoint.isBlank() || attempts < 0 || failedSubmissions < 0
45             || completedExercises < 0 || totalExercises < 0 || weaknessRate < 0 || weaknessRate > 1) {
46             throw new IllegalArgumentException("knowledge-point analytics values are invalid");
47         }
48     }
49 }
50 // FILE: src\main\java\com\sqlteacher\application\analytics\LearningAnalyticsReport.java
```

```
01 package com.sqlteacher.application.analytics;
02 import java.time.Instant;
03 import java.util.List;
04 import java.util.Objects;
05 public record LearningAnalyticsReport(
06     AnalyticsFilter filter,
07     Instant generatedAt,
08     AnalyticsOverview overview,
09     List<ExerciseAnalyticsRow> exercises,
10     List<ErrorAnalytics> commonErrors,
11     List<KnowledgePointAnalytics> knowledgePoints
12 ) {
13     public LearningAnalyticsReport {
14         Objects.requireNonNull(filter);
15         Objects.requireNonNull(generatedAt);
16         Objects.requireNonNull(overview);
17         exercises = List.copyOf(exercises);
18         commonErrors = List.copyOf(commonErrors);
19         knowledgePoints = List.copyOf(knowledgePoints);
20     }
21 }
22 // FILE: src\main\java\com\sqlteacher\application\analytics\LearningAnalyticsService.java
23 package com.sqlteacher.application.analytics;
24 public interface LearningAnalyticsService {
25     LearningAnalyticsReport analyze(AnalyticsFilter filter);
26     AnalyticsCsvExport exportCsv(AnalyticsFilter filter);
27 }
28 // FILE: src\main\java\com\sqlteacher\application\collaboration\AuthenticatedUser.java
29 package com.sqlteacher.application.collaboration;
30 import java.util.Objects;
31 import java.util.Set;
32 public record AuthenticatedUser(String id, String email, String displayName, Set<UserRole> roles) {
33     public AuthenticatedUser {
34         if (id == null || id.isBlank() || email == null || email.isBlank() || displayName == null ||
35         > displayName.isBlank()) {
36             throw new IllegalArgumentException("User fields must not be blank");
37         }
38         roles = Set.copyOf(Objects.requireNonNull(roles, "roles must not be null"));
39     }
40     public boolean hasRole(UserRole role) {
41         return roles.contains(Objects.requireNonNull(role, "role must not be null"));
42     }
43 }
44 // FILE: src\main\java\com\sqlteacher\application\collaboration\ClassAssignment.java
45 package com.sqlteacher.application.collaboration;
46 import java.time.Instant;
47 public record ClassAssignment(String id,String classroomId,String exerciseId,String title,Instant createdAt) {
48     public ClassAssignment {
49         if(id==null||id.isBlank()||classroomId==null||classroomId.isBlank()||exerciseId==null||exerciseId.isBlank()||t
50         > title==null||title.isBlank())throw new IllegalArgumentException("Assignment fields must not be blank");
```

```
01         if(createdAt==null)throw new IllegalArgumentException("createdAt must not be null");
02     }
03 }
04 // FILE: src\main\java\com\sqlteacher\application\collaboration\ClassroomService.java
05 package com.sqlteacher.application.collaboration;
06 import java.time.Instant;
07 import java.util.List;
08 import java.util.Objects;
09 public interface ClassroomService {
10     Classroom create(AuthenticatedUser actor, String name);
11     Classroom addMember(AuthenticatedUser actor, String classroomId, String userId, UserRole role);
12     List<Classroom> listVisibleTo(AuthenticatedUser actor);
13     record Classroom(String id, String name, Instant createdAt, List<Member> members) {
14         public Classroom {
15             if (id == null || id.isBlank() || name == null || name.isBlank()) {
16                 throw new IllegalArgumentException("Classroom fields must not be blank");
17             }
18             Objects.requireNonNull(createdAt, "createdAt must not be null");
19             members = List.copyOf(Objects.requireNonNull(members, "members must not be null"));
20         }
21     }
22     record Member(String userId, UserRole role) {
23         public Member {
24             if (userId == null || userId.isBlank()) {
25                 throw new IllegalArgumentException("userId must not be blank");
26             }
27             Objects.requireNonNull(role, "role must not be null");
28             if (role == UserRole.ADMIN) {
29                 throw new IllegalArgumentException("ADMIN is not a classroom membership role");
30             }
31         }
32     }
33 }
34 // FILE: src\main\java\com\sqlteacher\application\collaboration\CloudApiClient.java
35 package com.sqlteacher.application.collaboration;
36 import java.util.List;
37 /** Desktop boundary for the authenticated SQLTeacher cloud API. */
38 public interface CloudApiClient {
39     CloudAuthenticationService.Session login(String email, char[] password);
40     CloudAuthenticationService.Session register(String email, String displayName, char[] password);
41     void logout(String accessToken);
42     List<ClassroomService.Classroom> listClasses(String accessToken);
43     ClassroomService.Classroom createClass(String accessToken, String name);
44     ClassroomService.Classroom addClassMember(String accessToken, String classroomId, String email, UserRole role);
45     ClassAssignment createAssignment(String accessToken, String classroomId, String exerciseId, String title);
46     List<ClassAssignment> listAssignments(String accessToken, String classroomId);
47     int uploadSyncItems(String accessToken, List<CloudSyncItem> items);
48     List<CloudSyncItem> downloadSyncItems(String accessToken, long afterVersion);
49 }
50 // FILE: src\main\java\com\sqlteacher\application\collaboration\CloudAuthenticationService.java
```

```
01 package com.sqlteacher.application.collaboration;
02 import java.time.Instant;
03 import java.util.Objects;
04 /** Boundary used by clients and the cloud server; raw passwords never leave this use case. */
05 public interface CloudAuthenticationService {
06     Session register(String email, String displayName, char[] password);
07     Session login(String email, char[] password);
08     AuthenticatedUser authenticate(String accessToken);
09     void logout(String accessToken);
10     record Session(String accessToken, Instant expiresAt, AuthenticatedUser user) {
11         public Session {
12             if (accessToken == null || accessToken.isBlank()) {
13                 throw new IllegalArgumentException("accessToken must not be blank");
14             }
15             Objects.requireNonNull(expiresAt, "expiresAt must not be null");
16             Objects.requireNonNull(user, "user must not be null");
17         }
18     }
19 }
20 // FILE: src\main\java\com\sqlteacher\application\collaboration\CloudLearningSyncService.java
21 package com.sqlteacher.application.collaboration;
22 public interface CloudLearningSyncService {
23     SyncResult synchronize();
24     record SyncResult(int uploaded, int downloaded, long cursor) { }
25 }
26 // FILE: src\main\java\com\sqlteacher\application\collaboration\CloudSessionService.java
27 package com.sqlteacher.application.collaboration;
28 import java.util.Optional;
29 /** The desktop keeps only the active session in memory; no token is persisted to app.db. */
30 public interface CloudSessionService {
31     Optional<CloudAuthenticationService.Session> current();
32     void signIn(CloudAuthenticationService.Session session);
33     void signOut();
34 }
35 // FILE: src\main\java\com\sqlteacher\application\collaboration\CloudSyncItem.java
36 package com.sqlteacher.application.collaboration;
37 import java.time.Instant;
38 import java.util.Objects;
39 /** A user-scoped, idempotent learning record transferred to the cloud. */
40 public record CloudSyncItem(String id, String type, String payloadJson, Instant occurredAt, long version) {
41     public CloudSyncItem {
42         if (id == null || id.isBlank() || type == null || type.isBlank()) {
43             throw new IllegalArgumentException("Sync item id and type must not be blank");
44         }
45         Objects.requireNonNull(payloadJson, "payloadJson must not be null");
46         Objects.requireNonNull(occurredAt, "occurredAt must not be null");
47         if (version < 0) throw new IllegalArgumentException("version must not be negative");
48     }
49 }
50 // FILE: src\main\java\com\sqlteacher\application\collaboration\DesktopAccessProfile.java
```

```
01 package com.sqlteacher.application.collaboration;
02 import java.util.EnumSet;
03 import java.util.Objects;
04 import java.util.Set;
05 /** Effective desktop identity and its least-privilege navigation capabilities. */
06 public record DesktopAccessProfile(
07     Kind kind,
08     String userId,
09     String displayName,
10     String email,
11     Set<UserRole> serverRoles,
12     Set<DesktopCapability> capabilities
13 ) {
14     private static final Set<DesktopCapability> GUEST_CAPABILITIES = Set.copyOf(EnumSet.of(
15         DesktopCapability.HOME,
16         DesktopCapability.SQL_PRACTICE,
17         DesktopCapability.STUDENT_EXERCISE,
18         DesktopCapability.AI_ASSISTANT,
19         DesktopCapability.TABLE_SCHEMA
20     ));
21     private static final Set<DesktopCapability> STUDENT_CAPABILITIES = union(
22         GUEST_CAPABILITIES,
23         DesktopCapability.KNOWLEDGE_CENTER,
24         DesktopCapability.CLOUD_CENTER
25     );
26     private static final Set<DesktopCapability> TEACHER_CAPABILITIES = union(
27         STUDENT_CAPABILITIES,
28         DesktopCapability.EXERCISE_MANAGEMENT,
29         DesktopCapability.EXERCISE_PROGRESS,
30         DesktopCapability.SETTINGS
31     );
32     private static final Set<DesktopCapability> ADMIN_CAPABILITIES = Set.copyOf(
33         EnumSet.allOf(DesktopCapability.class)
34     );
35     public DesktopAccessProfile {
36         kind = Objects.requireNonNull(kind, "kind must not be null");
37         displayName = requireText(displayName, "displayName");
38         userId = userId == null ? "" : userId;
39         email = email == null ? "" : email;
40         serverRoles = Set.copyOf(Objects.requireNonNull(serverRoles, "serverRoles must not be null"));
41         capabilities = Set.copyOf(Objects.requireNonNull(capabilities, "capabilities must not be null"));
42         if (kind != Kind.GUEST && (userId.isBlank() || email.isBlank())) {
43             throw new IllegalArgumentException("Authenticated identity requires userId and email");
44         }
45     }
46     public static DesktopAccessProfile guest() {
47         return new DesktopAccessProfile(Kind.GUEST, "", "■■", "", Set.of(), GUEST_CAPABILITIES);
48     }
49     public static DesktopAccessProfile from(CloudAuthenticationService.Session session) {
50         Objects.requireNonNull(session, "session must not be null");
```

```
01     AuthenticatedUser user = session.user();
02     Kind kind;
03     Set<DesktopCapability> capabilities;
04     if (user.hasRole(UserRole.ADMIN)) {
05         kind = Kind.ADMIN;
06         capabilities = ADMIN_CAPABILITIES;
07     } else if (user.hasRole(UserRole.TEACHER)) {
08         kind = Kind.TEACHER;
09         capabilities = TEACHER_CAPABILITIES;
10     } else {
11         kind = Kind.STUDENT;
12         capabilities = STUDENT_CAPABILITIES;
13     }
14     return new DesktopAccessProfile(kind, user.id(), user.displayName(), user.email(), user.roles(),
15 > capabilities);
16 }
17 public boolean can(DesktopCapability capability) {
18     return capabilities.contains(Objects.requireNonNull(capability, "capability must not be null"));
19 }
20 public boolean isGuest() {
21     return kind == Kind.GUEST;
22 }
23 public String roleLabel() {
24     return switch (kind) {
25         case GUEST -> "■";
26         case STUDENT -> "■";
27         case TEACHER -> "■";
28         case ADMIN -> "■";
29     };
30 }
31 private static Set<DesktopCapability> union(
32     Set<DesktopCapability> base,
33     DesktopCapability... additions
34 ) {
35     EnumSet<DesktopCapability> result = EnumSet.copyOf(base);
36     result.addAll(Set.of(additions));
37     return Set.copyOf(result);
38 }
39 private static String requireText(String value, String name) {
40     if (value == null || value.isBlank()) throw new IllegalArgumentException(name + " must not be blank");
41     return value.trim();
42 }
43 public enum Kind {
44     GUEST,
45     STUDENT,
46     TEACHER,
47     ADMIN
48 }
49 }
50 // FILE: src\main\java\com\sqlteacher\application\collaboration\DesktopCapability.java
```

```
01 package com.sqlteacher.application.collaboration;
02 /** User-visible desktop modules controlled by the authenticated application identity. */
03 public enum DesktopCapability {
04     HOME,
05     SQL_PRACTICE,
06     STUDENT_EXERCISE,
07     EXERCISE_MANAGEMENT,
08     EXERCISE_PROGRESS,
09     KNOWLEDGE_CENTER,
10     AI_ASSISTANT,
11     TABLE_SCHEMA,
12     SETTINGS,
13     CLOUD_CENTER
14 }
15 // FILE: src\main\java\com\sqlteacher\application\collaboration\UserRole.java
16 package com.sqlteacher.application.collaboration;
17 public enum UserRole {
18     ADMIN,
19     TEACHER,
20     STUDENT
21 }
22 // FILE: src\main\java\com\sqlteacher\application\config\AiConfiguration.java
23 package com.sqlteacher.application.config;
24 import java.net.URI;
25 import java.time.Duration;
26 import java.util.Objects;
27 public record AiConfiguration(
28     URI ollamaBaseUrl,
29     Duration healthTimeout,
30     Duration generateTimeout,
31     String defaultModel
32 ) {
33     public AiConfiguration {
34         Objects.requireNonNull(ollamaBaseUrl, "ollamaBaseUrl must not be null");
35         Objects.requireNonNull(healthTimeout, "healthTimeout must not be null");
36         Objects.requireNonNull(generateTimeout, "generateTimeout must not be null");
37         Objects.requireNonNull(defaultModel, "defaultModel must not be null");
38         if (healthTimeout.isZero() || healthTimeout.isNegative()) {
39             throw new IllegalArgumentException("healthTimeout must be positive");
40         }
41         if (generateTimeout.isZero() || generateTimeout.isNegative()) {
42             throw new IllegalArgumentException("generateTimeout must be positive");
43         }
44         if (defaultModel.isBlank()) {
45             throw new IllegalArgumentException("defaultModel must not be blank");
46         }
47     }
48     public URI tagsEndpoint() {
49         return ollamaBaseUrl.resolve("/api/tags");
50     }
51 }
```

```
01     public URI generateEndpoint() {
02         return ollamaBaseUrl.resolve("/api/generate");
03     }
04 }
05 // FILE: src\main\java\com\sqlteacher\application\config\AppConfigurationService.java
06 package com.sqlteacher.application.config;
07 public interface AppConfigurationService {
08     SqlTeacherConfiguration current();
09 }
10 // FILE: src\main\java\com\sqlteacher\application\config\ApplicationVersion.java
11 package com.sqlteacher.application.config;
12 public final class ApplicationVersion {
13     private static final String DEVELOPMENT_VERSION = "development";
14     private ApplicationVersion() {
15     }
16     public static String current() {
17         String implementationVersion = ApplicationVersion.class.getPackage().getImplementationVersion();
18         return implementationVersion == null || implementationVersion.isBlank()
19             ? DEVELOPMENT_VERSION
20             : implementationVersion;
21     }
22 }
23 // FILE: src\main\java\com\sqlteacher\application\config\DatabaseConfiguration.java
24 package com.sqlteacher.application.config;
25 import java.nio.file.Path;
26 import java.util.Objects;
27 public record DatabaseConfiguration(
28     Path appDatabasePath,
29     Path demoDatabasePath
30 ) {
31     public DatabaseConfiguration {
32         Objects.requireNonNull(appDatabasePath, "appDatabasePath must not be null");
33         Objects.requireNonNull(demoDatabasePath, "demoDatabasePath must not be null");
34     }
35 }
36 // FILE: src\main\java\com\sqlteacher\application\config\SqlTeacherConfiguration.java
37 package com.sqlteacher.application.config;
38 import java.nio.file.Path;
39 import java.util.Objects;
40 public record SqlTeacherConfiguration(
41     String appName,
42     Path dataDirectory,
43     DatabaseConfiguration database,
44     AiConfiguration ai
45 ) {
46     public SqlTeacherConfiguration {
47         if (appName == null || appName.isBlank()) {
48             throw new IllegalArgumentException("appName must not be blank");
49         }
50         Objects.requireNonNull(dataDirectory, "dataDirectory must not be null");
```

```
01         Objects.requireNonNull(database, "database must not be null");
02         Objects.requireNonNull(ai, "ai must not be null");
03     }
04 }
05 // FILE: src\main\java\com\sqlteacher\application\connection\ConnectionManagementService.java
06 package com.sqlteacher.application.connection;
07 import java.util.List;
08 import java.util.Optional;
09 /** Manages non-sensitive connection profiles and the connection selected for application use cases. */
10 public interface ConnectionManagementService {
11     List<DatabaseConnectionProfile> listProfiles();
12     Optional<DatabaseConnectionProfile> findProfile(String connectionId);
13     DatabaseConnectionProfile saveProfile(DatabaseConnectionProfile profile);
14     void removeProfile(String connectionId);
15     Optional<DatabaseConnectionProfile> currentProfile();
16     DatabaseConnectionProfile selectProfile(String connectionId);
17 }
18 // FILE: src\main\java\com\sqlteacher\application\connection\DatabaseConnectionProfile.java
19 package com.sqlteacher.application.connection;
20 import java.util.Objects;
21 import java.util.regex.Pattern;
22 public record DatabaseConnectionProfile(
23     String id,
24     String displayName,
25     DatabaseConnectionTarget target,
26     boolean readOnly,
27     boolean enabled,
28     boolean builtIn
29 ) {
30     private static final Pattern ID_PATTERN = Pattern.compile("[a-z0-9][a-z0-9._-]{0,63}");
31     public DatabaseConnectionProfile {
32         id = requireText(id, "id");
33         if (!ID_PATTERN.matcher(id).matches()) {
34             throw new IllegalArgumentException(
35                 "id must contain 1-64 lowercase letters, digits, dots, underscores or hyphens"
36             );
37         }
38         displayName = requireText(displayName, "displayName");
39         Objects.requireNonNull(target, "target must not be null");
40     }
41     public DatabaseDialect dialect() {
42         return target.dialect();
43     }
44     private static String requireText(String value, String fieldName) {
45         Objects.requireNonNull(value, fieldName + " must not be null");
46         String normalized = value.trim();
47         if (normalized.isEmpty()) {
48             throw new IllegalArgumentException(fieldName + " must not be blank");
49         }
50         return normalized;
```

```
01     }
02 }
03 // FILE: src\main\java\com\sqlteacher\application\connection\DatabaseConnectionTarget.java
04 package com.sqlteacher.application.connection;
05 public sealed interface DatabaseConnectionTarget
06     permits SQLiteConnectionTarget, ServerConnectionTarget {
07     DatabaseDialect dialect();
08 }
09 // FILE: src\main\java\com\sqlteacher\application\connection\DatabaseConnectionTestResult.java
10 package com.sqlteacher.application.connection;
11 import java.time.Duration;
12 import java.util.Objects;
13 public record DatabaseConnectionTestResult(
14     boolean successful,
15     String message,
16     String databaseProduct,
17     String databaseVersion,
18     Duration elapsed
19 ) {
20     public DatabaseConnectionTestResult {
21         message = requireText(message, "message");
22         databaseProduct = Objects.requireNonNull(databaseProduct, "databaseProduct must not be null").trim();
23         databaseVersion = Objects.requireNonNull(databaseVersion, "databaseVersion must not be null").trim();
24         Objects.requireNonNull(elapsed, "elapsed must not be null");
25         if (elapsed.isNegative()) {
26             throw new IllegalArgumentException("elapsed must not be negative");
27         }
28     }
29     private static String requireText(String value, String fieldName) {
30         Objects.requireNonNull(value, fieldName + " must not be null");
31         String normalized = value.trim();
32         if (normalized.isEmpty()) {
33             throw new IllegalArgumentException(fieldName + " must not be blank");
34         }
35         return normalized;
36     }
37 }
38 // FILE: src\main\java\com\sqlteacher\application\connection\DatabaseConnectionTestService.java
39 package com.sqlteacher.application.connection;
40 /** Tests a connection without exposing JDBC details to desktop callers. */
41 public interface DatabaseConnectionTestService {
42     /**
43      * Tests the supplied profile with a transient password.
44      *
45      * <p>The implementation must not retain, log, or include the password in its result. The caller owns the
46      * array and should clear it after this method returns.</p>
47      */
48     DatabaseConnectionTestResult testConnection(DatabaseConnectionProfile profile, char[] password);
49 }
50 // FILE: src\main\java\com\sqlteacher\application\connection\DatabaseCredentialSession.java
```

```
01 package com.sqlteacher.application.connection;
02 import java.util.Optional;
03 /** Holds database passwords in memory for the current application process only. */
04 public interface DatabaseCredentialSession extends AutoCloseable {
05     void remember(String connectionId, char[] password);
06     Optional<char[]> passwordFor(String connectionId);
07     void forget(String connectionId);
08     @Override
09     void close();
10 }
11 // FILE: src\main\java\com\sqlteacher\application\connection\DatabaseDialect.java
12 package com.sqlteacher.application.connection;
13 public enum DatabaseDialect {
14     SQLITE,
15     MYSQL,
16     MARIADB
17 }
18 // FILE: src\main\java\com\sqlteacher\application\connection\ServerConnectionTarget.java
19 package com.sqlteacher.application.connection;
20 import java.util.Objects;
21 public record ServerConnectionTarget(
22     DatabaseDialect dialect,
23     String host,
24     int port,
25     String databaseName,
26     String username
27 ) implements DatabaseConnectionTarget {
28     public ServerConnectionTarget {
29         Objects.requireNonNull(dialect, "dialect must not be null");
30         if (dialect == DatabaseDialect.SQLITE) {
31             throw new IllegalArgumentException("Server connection target requires MySQL or MariaDB dialect");
32         }
33         host = requireText(host, "host");
34         if (port < 1 || port > 65535) {
35             throw new IllegalArgumentException("port must be between 1 and 65535");
36         }
37         databaseName = requireText(databaseName, "databaseName");
38         username = requireText(username, "username");
39     }
40     private static String requireText(String value, String fieldName) {
41         Objects.requireNonNull(value, fieldName + " must not be null");
42         String normalized = value.trim();
43         if (normalized.isEmpty()) {
44             throw new IllegalArgumentException(fieldName + " must not be blank");
45         }
46         return normalized;
47     }
48 }
49 // FILE: src\main\java\com\sqlteacher\application\connection\SqliteConnectionTarget.java
50 package com.sqlteacher.application.connection;
```

```
01 import java.nio.file.Path;
02 import java.util.Objects;
03 public record SqliteConnectionTarget(Path databasePath) implements DatabaseConnectionTarget {
04     public SqliteConnectionTarget {
05         Objects.requireNonNull(databasePath, "databasePath must not be null");
06         databasePath = databasePath.normalize();
07     }
08     @Override
09     public DatabaseDialect dialect() {
10         return DatabaseDialect.SQLITE;
11     }
12 }
13 // FILE: src\main\java\com\sqlteacher\application\database\DatabaseInitializationResult.java
14 package com.sqlteacher.application.database;
15 import java.nio.file.Path;
16 public record DatabaseInitializationResult(
17     Path appDatabasePath,
18     Path demoDatabasePath,
19     boolean appDatabaseCreated,
20     boolean demoDatabaseCreated
21 ) {
22 }
23 // FILE: src\main\java\com\sqlteacher\application\database\DatabaseInitializationService.java
24 package com.sqlteacher.application.database;
25 public interface DatabaseInitializationService {
26     DatabaseInitializationResult initialize();
27 }
28 // FILE: src\main\java\com\sqlteacher\application\error\ApplicationError.java
29 package com.sqlteacher.application.error;
30 import java.util.Objects;
31 public record ApplicationError(
32     String code,
33     ApplicationErrorType type,
34     String userMessage,
35     boolean retryable
36 ) {
37     public ApplicationError {
38         if (code == null || code.isBlank()) {
39             throw new IllegalArgumentException("code must not be blank");
40         }
41         Objects.requireNonNull(type, "type must not be null");
42         if (userMessage == null || userMessage.isBlank()) {
43             throw new IllegalArgumentException("userMessage must not be blank");
44         }
45     }
46 }
47 // FILE: src\main\java\com\sqlteacher\application\error\ApplicationErrorType.java
48 package com.sqlteacher.application.error;
49 public enum ApplicationErrorType {
50     VALIDATION,
```

```
01     APPLICATION,
02     UNEXPECTED
03 }
04 // FILE: src\main\java\com\sqlteacher\application\error\ApplicationExceptionMapper.java
05 package com.sqlteacher.application.error;
06 public interface ApplicationExceptionMapper {
07     ApplicationError map(Throwable throwable);
08 }
09 // FILE: src\main\java\com\sqlteacher\application\error\DefaultApplicationExceptionMapper.java
10 package com.sqlteacher.application.error;
11 import com.sqlteacher.domain.SqlTeacherException;
12 import org.slf4j.Logger;
13 import org.slf4j.LoggerFactory;
14 import java.util.Map;
15 import java.util.Objects;
16 public final class DefaultApplicationExceptionMapper implements ApplicationExceptionMapper {
17     private static final Logger log = LoggerFactory.getLogger(DefaultApplicationExceptionMapper.class);
18     private static final Map<String, ErrorPresentation> PRESENTATIONS = Map.ofEntries(
19         Map.entry("CONFIG_NOT_FOUND", presentation("■■■■■■■■■■■■■■■■■■■■", false)),
20         Map.entry("CONFIG_LOAD_FAILED", presentation("■■■■■■■■■■■■■■■■■■■■", true)),
21         Map.entry("CONFIG_INVALID", presentation("■■■■■■■■■■■■■■■■■■■■", false)),
22         Map.entry("SQLITE_INIT_FAILED", presentation("■■■■■■■■■■■■■■■■■■■■", true)),
23         Map.entry("CONNECTION_PROFILE_FAILED", presentation("■■■■■■■■■■■■■■■■■■■■", true)),
24         Map.entry("DATABASE_CONNECTION_NOT_FOUND", presentation("■■■■■■■■■■■■■■■■■■■■", false)),
25         Map.entry("DATABASE_CONNECTION_DISABLED", presentation("■■■■■■■■■■■■■■■■■■■■", false)),
26         Map.entry("DATABASE_CREDENTIALS_REQUIRED", presentation("■■■■■■■■■■■■■■■■■■■■", false)),
27         Map.entry("DATABASE_AUTHENTICATION_FAILED", presentation("■■■■■■■■■■■■■■■■■■■■", false)),
28         Map.entry("DATABASE_PERMISSION_DENIED", presentation("■■■■■■■■■■■■■■■■■■■■", false)),
29         Map.entry("DATABASE_CONNECTION_TIMEOUT", presentation("■■■■■■■■■■■■■■■■■■■■", true)),
30         Map.entry("DATABASE_CONNECTION_FAILED", presentation("■■■■■■■■■■■■■■■■■■■■", true)),
31         Map.entry("EXTERNAL_CONNECTION_READ_ONLY_REQUIRED", presentation("■■■■■■■■■■■■■■■■■■■■", false)),
32         Map.entry("DATABASE_METADATA_FAILED", presentation("■■■■■■■■■■■■■■■■■■■■", true)),
33         Map.entry("SQL_BLOCKED", presentation("SQL ■■■■■■■■■■■■■■■■■■■■■", false)),
34         Map.entry("SQL_READ_ONLY_CONNECTION", presentation("■■■■■■■■■■■■■■■■■■■■ SELECT ■■■", false)),
35         Map.entry("MOCK_SQL_BLOCKED", presentation("SQL ■■■■■■■■■■■■■■■■■■■■■", false)),
36         Map.entry("SQL_CONFIRMATION_REQUIRED", presentation("■ SQL ■■■■■■■■■■■■■■■■■■■■■", false)),
37         Map.entry("SQL_EXECUTION_FAILED", presentation("SQL ■■■■■■■■■■■■■■■■■■■■■", true))
38     );
39     @Override
40     public ApplicationError map(Throwable throwable) {
41         Objects.requireNonNull(throwable, "throwable must not be null");
42         if (throwable instanceof SqlTeacherException exception) {
43             String code = hasText(exception.errorCode()) ? exception.errorCode() : "APPLICATION_ERROR";
44             ErrorPresentation presentation = PRESENTATIONS.get(code);
45             String userMessage = presentation != null
46                 ? presentation.userMessage()
47                 : "■■■■■■■■■■■■■■■■■■■■";
48             boolean retryable = presentation != null && presentation.retryable();
49             log.warn(
50                 "Application operation failed, code={}, exceptionType={}",
```

[illegible]

```
01 public final class DefaultLearningEventService implements LearningEventService {
02     private final LearningEventRecorder recorder;
03     private final Clock clock;
04     private final LearningEventOwnerProvider ownerProvider;
05     public DefaultLearningEventService(LearningEventRecorder recorder) {
06         this(recorder, Clock.systemUTC(), () -> LearningEventOwnerProvider.GUEST_OWNER);
07     }
08     public DefaultLearningEventService(
09         LearningEventRecorder recorder,
10         LearningEventOwnerProvider ownerProvider
11     ) {
12         this(recorder, Clock.systemUTC(), ownerProvider);
13     }
14     DefaultLearningEventService(LearningEventRecorder recorder, Clock clock) {
15         this(recorder, clock, () -> LearningEventOwnerProvider.GUEST_OWNER);
16     }
17     DefaultLearningEventService(
18         LearningEventRecorder recorder,
19         Clock clock,
20         LearningEventOwnerProvider ownerProvider
21     ) {
22         this.recorder = Objects.requireNonNull(recorder, "recorder must not be null");
23         this.clock = Objects.requireNonNull(clock, "clock must not be null");
24         this.ownerProvider = Objects.requireNonNull(ownerProvider, "ownerProvider must not be null");
25     }
26     @Override
27     public void recordSqlExecution(
28         String connectionId,
29         boolean successful,
30         String statementType,
31         Duration duration,
32         int resultCount,
33         String errorCode
34     ) {
35         validateConnectionId(connectionId);
36         validateText(statementType, "statementType");
37         Objects.requireNonNull(duration, "duration must not be null");
38         if (duration.isNegative()) {
39             throw new IllegalArgumentException("duration must not be negative");
40         }
41         if (resultCount < 0) {
42             throw new IllegalArgumentException("resultCount must not be negative");
43         }
44         Map<String, String> attributes = new LinkedHashMap<>();
45         attributes.put("statementType", statementType);
46         attributes.put("durationMs", Long.toString(duration.toMillis()));
47         attributes.put("resultCount", Integer.toString(resultCount));
48         putIfPresent(attributes, "errorCode", errorCode);
49         record(LearningEventType.SQL_EXECUTION, connectionId, successful, attributes);
50     }
```

```
01     @Override
02     public void recordSqlRiskBlocked(
03         String connectionId,
04         String statementType,
05         SqlRiskLevel riskLevel,
06         boolean multiStatement
07     ) {
08         validateConnectionId(connectionId);
09         validateText(statementType, "statementType");
10         Objects.requireNonNull(riskLevel, "riskLevel must not be null");
11         record(
12             LearningEventType.SQL_RISK_BLOCKED,
13             connectionId,
14             false,
15             Map.of(
16                 "statementType", statementType,
17                 "riskLevel", riskLevel.name(),
18                 "multiStatement", Boolean.toString(multiStatement)
19             )
20         );
21     }
22     @Override
23     public void recordAiGeneration(
24         String connectionId,
25         boolean successful,
26         String model,
27         String promptVersion,
28         String errorCode
29     ) {
30         validateConnectionId(connectionId);
31         validateText(model, "model");
32         validateText(promptVersion, "promptVersion");
33         Map<String, String> attributes = new LinkedHashMap<>();
34         attributes.put("model", model);
35         attributes.put("promptVersion", promptVersion);
36         putIfPresent(attributes, "errorCode", errorCode);
37         record(
38             successful ? LearningEventType.AI_SQL_GENERATED : LearningEventType.AI_GENERATION_FAILED,
39             connectionId,
40             successful,
41             attributes
42         );
43     }
44     @Override
45     public void recordExerciseAttempt(
46         String exerciseId,
47         String status,
48         boolean successful,
49         Duration duration,
50         String errorCode
```

```
01     ) {
02         validateText(exerciseId, "exerciseId");
03         validateText(status, "status");
04         Objects.requireNonNull(duration, "duration must not be null");
05         if (duration.isNegative()) {
06             throw new IllegalArgumentException("duration must not be negative");
07         }
08         Map<String, String> attributes = new LinkedHashMap<>();
09         attributes.put("exerciseId", exerciseId);
10         attributes.put("status", status);
11         attributes.put("durationMs", Long.toString(duration.toMillis()));
12         putIfPresent(attributes, "errorCode", errorCode);
13         LearningEventType type = switch (status) {
14             case "PASSED" -> LearningEventType.EXERCISE_PASSED;
15             case "FAILED" -> LearningEventType.EXERCISE_FAILED;
16             default -> LearningEventType.EXERCISE_ATTEMPT;
17         };
18         record(type, "exercise", successful, attributes);
19     }
20     @Override
21     public void recordExerciseHint(String exerciseId, int hintLevel) {
22         validateText(exerciseId, "exerciseId");
23         if (hintLevel < 1) {
24             throw new IllegalArgumentException("hintLevel must be positive");
25         }
26         record(
27             LearningEventType.EXERCISE_HINT_USED,
28             "exercise",
29             true,
30             Map.of("exerciseId", exerciseId, "hintLevel", Integer.toString(hintLevel))
31         );
32     }
33     @Override
34     public void recordKnowledgeSearch(int queryLength, int resultCount) {
35         if (queryLength < 1 || resultCount < 0) {
36             throw new IllegalArgumentException("knowledge search metrics are invalid");
37         }
38         record(
39             LearningEventType.KNOWLEDGE_SEARCHED,
40             "knowledge",
41             true,
42             Map.of("queryLength", Integer.toString(queryLength), "resultCount", Integer.toString(resultCount))
43         );
44     }
45     private void record(
46         LearningEventType type,
47         String connectionId,
48         boolean successful,
49         Map<String, String> attributes
50     ) {
```

```
01     Map<String, String> ownedAttributes = new LinkedHashMap<>(attributes);
02     ownedAttributes.put(LearningEventOwnerProvider.OWNER_ATTRIBUTE, ownerProvider.currentOwnerId());
03     recorder.record(new LearningEvent(type, clock.instant(), connectionId, successful, ownedAttributes));
04 }
05 private static void validateConnectionId(String connectionId) {
06     validateText(connectionId, "connectionId");
07 }
08 private static void validateText(String value, String name) {
09     if (value == null || value.isBlank()) {
10         throw new IllegalArgumentException(name + " must not be blank");
11     }
12 }
13 private static void putIfPresent(Map<String, String> attributes, String name, String value) {
14     if (value != null && !value.isBlank()) {
15         attributes.put(name, value);
16     }
17 }
18 }
19 // FILE: src\main\java\com\sqlteacher\application\event\LearningEvent.java
20 package com.sqlteacher.application.event;
21 import java.time.Instant;
22 import java.util.Map;
23 import java.util.Objects;
24 public record LearningEvent(
25     LearningEventType type,
26     Instant occurredAt,
27     String connectionId,
28     boolean successful,
29     Map<String, String> attributes
30 ) {
31     public LearningEvent {
32         Objects.requireNonNull(type, "type must not be null");
33         Objects.requireNonNull(occurredAt, "occurredAt must not be null");
34         if (connectionId == null || connectionId.isBlank()) {
35             throw new IllegalArgumentException("connectionId must not be blank");
36         }
37         Objects.requireNonNull(attributes, "attributes must not be null");
38         attributes = Map.copyOf(attributes);
39     }
40 }
41 // FILE: src\main\java\com\sqlteacher\application\event\LearningEventOwnerProvider.java
42 package com.sqlteacher.application.event;
43 /** Supplies the local identity that owns newly recorded learning events. */
44 @FunctionalInterface
45 public interface LearningEventOwnerProvider {
46     String OWNER_ATTRIBUTE = "_desktop_owner_id";
47     String GUEST_OWNER = "guest";
48     String currentOwnerId();
49 }
50 // FILE: src\main\java\com\sqlteacher\application\event\LearningEventQueryService.java
```

```
01 package com.sqlteacher.application.event;
02 import java.time.Instant;
03 import java.util.List;
04 import java.util.Map;
05 import java.util.Objects;
06 /**
07  * Query service for learning events to support analytics and debugging.
08  */
09 public interface LearningEventQueryService {
10     /**
11      * Query events by type within a time range.
12      *
13      * @param type the event type to filter by
14      * @param start the start of the time range (inclusive), or null for no lower bound
15      * @param end the end of the time range (inclusive), or null for no upper bound
16      * @return list of matching events, never null
17      */
18     List<QueriedLearningEvent> queryEventsByType(LearningEventType type, Instant start, Instant end);
19     /**
20      * Query events by connection ID within a time range.
21      *
22      * @param connectionId the connection ID to filter by
23      * @param start the start of the time range (inclusive), or null for no lower bound
24      * @param end the end of the time range (inclusive), or null for no upper bound
25      * @return list of matching events, never null
26      */
27     List<QueriedLearningEvent> queryEventsByConnection(String connectionId, Instant start, Instant end);
28     /**
29      * Get event statistics within a time range.
30      *
31      * @param start the start of the time range (inclusive), or null for no lower bound
32      * @param end the end of the time range (inclusive), or null for no upper bound
33      * @return statistics about events in the time range
34      */
35     EventStatistics getEventStatistics(Instant start, Instant end);
36     /**
37      * A read-only representation of a queried learning event.
38      */
39     record QueriedLearningEvent(
40         long id,
41         LearningEventType type,
42         Instant occurredAt,
43         String connectionId,
44         boolean successful,
45         Map<String, String> attributes,
46         Instant createdAt
47     ) {
48         public QueriedLearningEvent {
49             Objects.requireNonNull(type, "type must not be null");
50             Objects.requireNonNull(occurredAt, "occurredAt must not be null");
```

```
01         Objects.requireNonNull(connectionId, "connectionId must not be null");
02         attributes = Map.copyOf(Objects.requireNonNull(attributes, "attributes must not be null"));
03         Objects.requireNonNull(createdAt, "createdAt must not be null");
04     }
05 }
06 /**
07  * Statistics about learning events in a time range.
08  */
09 record EventStatistics(
10     long totalEvents,
11     long successfulEvents,
12     long failedEvents,
13     Map<LearningEventType, Long> eventsByType,
14     Map<String, Long> eventsByConnection
15 ) {
16     public EventStatistics {
17         eventsByType = Map.copyOf(Objects.requireNonNull(eventsByType, "eventsByType must not be null"));
18         eventsByConnection = Map.copyOf(
19             Objects.requireNonNull(eventsByConnection, "eventsByConnection must not be null"));
20     }
21 }
22 }
23 // FILE: src/main/java/com/sqlteacher/application/event/LearningEventRecorder.java
24 package com.sqlteacher.application.event;
25 public interface LearningEventRecorder {
26     /**
27      * Persists an application event. Implementations own all storage details and must not
28      * require callers to know a table, file, or serialization format.
29      */
30     void record(LearningEvent event);
31 }
32 // FILE: src/main/java/com/sqlteacher/application/event/LearningEventService.java
33 package com.sqlteacher.application.event;
34 import com.sqlteacher.application.risk.SqlRiskLevel;
35 import java.time.Duration;
36 /**
37  * Records P0 learning events without accepting raw SQL, natural-language prompts, or model output.
38  */
39 public interface LearningEventService {
40     void recordSqlExecution(
41         String connectionId,
42         boolean successful,
43         String statementType,
44         Duration duration,
45         int resultCount,
46         String errorCode
47     );
48     void recordSqlRiskBlocked(
49         String connectionId,
50         String statementType,
```

```
01         SqlRiskLevel riskLevel,
02         boolean multiStatement
03     );
04     void recordAiGeneration(
05         String connectionId,
06         boolean successful,
07         String model,
08         String promptVersion,
09         String errorCode
10     );
11     default void recordExerciseAttempt(
12         String exerciseId,
13         String status,
14         boolean successful,
15         Duration duration,
16         String errorCode
17     ) {
18     }
19     default void recordExerciseHint(String exerciseId, int hintLevel) {
20     }
21     default void recordKnowledgeSearch(int queryLength, int resultCount) {
22     }
23 }
24 // FILE: src\main\java\com\sqlteacher\application\event\LearningEventType.java
25 package com.sqlteacher.application.event;
26 public enum LearningEventType {
27     SQL_EXECUTION,
28     SQL_RISK_BLOCKED,
29     AI_SQL_GENERATED,
30     AI_GENERATION_FAILED,
31     EXERCISE_ATTEMPT,
32     EXERCISE_PASSED,
33     EXERCISE_FAILED,
34     EXERCISE_HINT_USED,
35     KNOWLEDGE_SEARCHED
36 }
37 // FILE: src\main\java\com\sqlteacher\application\execution\SqlExecutionRequest.java
38 package com.sqlteacher.application.execution;
39 import java.time.Duration;
40 public record SqlExecutionRequest(
41     String connectionId,
42     String sql,
43     int maxRows,
44     Duration timeout,
45     boolean riskConfirmed
46 ) {
47     public SqlExecutionRequest(String connectionId, String sql, int maxRows, Duration timeout) {
48         this(connectionId, sql, maxRows, timeout, false);
49     }
50 }
```

```
01 // FILE: src\main\java\com\sqlteacher\application\execution\SqlExecutionResult.java
02 package com.sqlteacher.application.execution;
03 import java.util.List;
04 import java.util.Map;
05 import java.time.Duration;
06 import java.util.Collections;
07 import java.util.LinkedHashMap;
08 public record SqlExecutionResult(
09     boolean success,
10     List<String> columns,
11     List<Map<String, Object>> rows,
12     int affectedRows,
13     boolean truncated,
14     String message,
15     Duration duration
16 ) {
17     public SqlExecutionResult {
18         columns = List.copyOf(columns);
19         rows = rows.stream()
20             .map(row -> Collections.unmodifiableMap(new LinkedHashMap<>(row)))
21             .toList();
22     }
23     public SqlExecutionResult(
24         boolean success,
25         List<String> columns,
26         List<Map<String, Object>> rows,
27         String message
28     ) {
29         this(success, columns, rows, 0, false, message, Duration.ZERO);
30     }
31 }
32 // FILE: src\main\java\com\sqlteacher\application\execution\SqlExecutionService.java
33 package com.sqlteacher.application.execution;
34 public interface SqlExecutionService {
35     SqlExecutionResult execute(SqlExecutionRequest request);
36 }
37 // FILE: src\main\java\com\sqlteacher\application\exercise\EvaluationCriterionResult.java
38 package com.sqlteacher.application.exercise;
39 import java.util.Objects;
40 public record EvaluationCriterionResult(
41     String criterion,
42     boolean passed,
43     String feedback
44 ) {
45     public EvaluationCriterionResult {
46         criterion = requireText(criterion, "criterion");
47         feedback = requireText(feedback, "feedback");
48     }
49     private static String requireText(String value, String fieldName) {
50         Objects.requireNonNull(value, fieldName + " must not be null");
51     }
52 }
```

```
01         String normalized = value.trim();
02         if (normalized.isEmpty()) {
03             throw new IllegalArgumentException(fieldName + " must not be blank");
04         }
05         return normalized;
06     }
07 }
08 // FILE: src\main\java\com\sqlteacher\application\exercise\ExerciseAttemptResult.java
09 package com.sqlteacher.application.exercise;
10 import com.sqlteacher.application.execution.SqlExecutionResult;
11 import com.sqlteacher.domain.exercise.ExerciseAttemptStatus;
12 import java.time.Instant;
13 import java.util.Objects;
14 import java.util.Optional;
15 public record ExerciseAttemptResult(
16     String attemptId,
17     String sessionId,
18     ExerciseAttemptStatus status,
19     SqlExecutionResult execution,
20     ExerciseEvaluationResult evaluation,
21     Instant occurredAt
22 ) {
23     public ExerciseAttemptResult {
24         attemptId = requireText(attemptId, "attemptId");
25         sessionId = requireText(sessionId, "sessionId");
26         status = Objects.requireNonNull(status, "status must not be null");
27         occurredAt = Objects.requireNonNull(occurredAt, "occurredAt must not be null");
28         if ((status == ExerciseAttemptStatus.PASSED || status == ExerciseAttemptStatus.FAILED)
29             && evaluation == null) {
30             throw new IllegalArgumentException("submitted attempts require evaluation");
31         }
32     }
33     public Optional<SqlExecutionResult> executionResult() {
34         return Optional.ofNullable(execution);
35     }
36     public Optional<ExerciseEvaluationResult> evaluationResult() {
37         return Optional.ofNullable(evaluation);
38     }
39     private static String requireText(String value, String fieldName) {
40         Objects.requireNonNull(value, fieldName + " must not be null");
41         String normalized = value.trim();
42         if (normalized.isEmpty()) {
43             throw new IllegalArgumentException(fieldName + " must not be blank");
44         }
45         return normalized;
46     }
47 }
48 // FILE: src\main\java\com\sqlteacher\application\exercise\ExerciseCatalogService.java
49 package com.sqlteacher.application.exercise;
50 import java.util.List;
```

```
01 import java.util.Optional;
02 public interface ExerciseCatalogService {
03     List<ExerciseSummary> listAvailableExercises();
04     Optional<ExerciseView> findAvailableExercise(String exerciseId);
05 }
06 // FILE: src\main\java\com\sqlteacher\application\exercise\ExerciseDraft.java
07 package com.sqlteacher.application.exercise;
08 import com.sqlteacher.domain.exercise.ExerciseDifficulty;
09 import com.sqlteacher.domain.exercise.ExerciseEvaluationRule;
10 import java.util.List;
11 import java.util.Objects;
12 public record ExerciseDraft(
13     String id,
14     String title,
15     String description,
16     String knowledgePoint,
17     ExerciseDifficulty difficulty,
18     String datasetId,
19     String referenceSql,
20     ExerciseEvaluationRule evaluationRule,
21     List<String> hints,
22     Integer expectedVersion,
23     boolean enabled
24 ) {
25     public ExerciseDraft {
26         id = normalizeOptionalId(id);
27         title = requireText(title, "title");
28         description = requireText(description, "description");
29         knowledgePoint = requireText(knowledgePoint, "knowledgePoint");
30         difficulty = Objects.requireNonNull(difficulty, "difficulty must not be null");
31         datasetId = requireText(datasetId, "datasetId");
32         referenceSql = requireText(referenceSql, "referenceSql");
33         evaluationRule = Objects.requireNonNull(evaluationRule, "evaluationRule must not be null");
34         hints = List.copyOf(Objects.requireNonNull(hints, "hints must not be null"));
35         if (expectedVersion != null && expectedVersion < 1) {
36             throw new IllegalArgumentException("expectedVersion must be positive");
37         }
38     }
39     private static String normalizeOptionalId(String value) {
40         return value == null || value.isBlank() ? "" : value.trim();
41     }
42     private static String requireText(String value, String fieldName) {
43         Objects.requireNonNull(value, fieldName + " must not be null");
44         String normalized = value.trim();
45         if (normalized.isEmpty()) {
46             throw new IllegalArgumentException(fieldName + " must not be blank");
47         }
48         return normalized;
49     }
50 }
```

```
01 // FILE: src\main\java\com\sqlteacher\application\exercise\ExerciseEvaluationResult.java
02 package com.sqlteacher.application.exercise;
03 import java.time.Duration;
04 import java.util.List;
05 import java.util.Objects;
06 public record ExerciseEvaluationResult(
07     boolean passed,
08     List<EvaluationCriterionResult> criteria,
09     String feedback,
10     Duration duration,
11     String errorCode
12 ) {
13     public ExerciseEvaluationResult {
14         criteria = List.copyOf(Objects.requireNonNull(criteria, "criteria must not be null"));
15         feedback = Objects.requireNonNull(feedback, "feedback must not be null").trim();
16         duration = Objects.requireNonNull(duration, "duration must not be null");
17         errorCode = errorCode == null ? "" : errorCode.trim();
18         if (duration.isNegative()) {
19             throw new IllegalArgumentException("duration must not be negative");
20         }
21         if (passed && criteria.stream().anyMatch(criterion -> !criterion.passed())) {
22             throw new IllegalArgumentException("passed result cannot contain failed criteria");
23         }
24     }
25 }
26 // FILE: src\main\java\com\sqlteacher\application\exercise\ExerciseHint.java
27 package com.sqlteacher.application.exercise;
28 import java.util.Objects;
29 public record ExerciseHint(int level, String text, boolean exhausted) {
30     public ExerciseHint {
31         if (level < 1 || level > 3) {
32             throw new IllegalArgumentException("level must be between 1 and 3");
33         }
34         text = Objects.requireNonNull(text, "text must not be null").trim();
35         if (text.isEmpty()) {
36             throw new IllegalArgumentException("text must not be blank");
37         }
38     }
39 }
40 // FILE: src\main\java\com\sqlteacher\application\exercise\ExerciseImportResult.java
41 package com.sqlteacher.application.exercise;
42 import java.util.List;
43 import java.util.Objects;
44 public record ExerciseImportResult(
45     int datasetsImported,
46     int exercisesImported,
47     List<String> importedExerciseIds
48 ) {
49     public ExerciseImportResult {
50         if (datasetsImported < 0 || exercisesImported < 0) {
```

```
01         values (?, ?, ?, ?, ?, ?)
02     """)) {
03         statement.setString(1, id);
04         statement.setString(2, title);
05         statement.setString(3, sourceName);
06         statement.setString(4, hash);
07         statement.setInt(5, chunkCount);
08         statement.setString(6, importedAt.toString());
09         statement.executeUpdate();
10     }
11 }
12 private static void insertChunks(Connection connection, String documentId, List<String> chunks) throws
13 > SQLException {
14     try (PreparedStatement statement = connection.prepareStatement("""
15         insert into knowledge_chunks(id, document_id, chunk_index, content) values (?, ?, ?, ?)
16         """)) {
17         for (int index = 0; index < chunks.size(); index++) {
18             statement.setString(1, UUID.randomUUID().toString());
19             statement.setString(2, documentId);
20             statement.setInt(3, index);
21             statement.setString(4, chunks.get(index));
22             statement.addBatch();
23         }
24         statement.executeBatch();
25     }
26 }
27 private static String title(Path path, String content) {
28     for (String line : content.lines().limit(20).toList()) {
29         String trimmed = line.trim();
30         if (trimmed.startsWith("#")) {
31             String heading = trimmed.replaceFirst("^#+\\s*", "").trim();
32             if (!heading.isBlank()) {
33                 return truncate(heading, 160);
34             }
35         }
36     }
37     String fileName = path.getFileName().toString();
38     int dot = fileName.lastIndexOf('.');
39     return truncate(dot > 0 ? fileName.substring(0, dot) : fileName, 160);
40 }
41 private static String toFtsQuery(String query) {
42     String[] tokens = query.trim().split("\\s+");
43     List<String> phrases = new ArrayList<>();
44     for (String token : tokens) {
45         String safe = token.replace("\"", "\\\"").trim();
46         if (!safe.isBlank()) {
47             phrases.add("\"" + safe + "\"");
48         }
49     }
50     if (phrases.isEmpty()) {
```

```
01         throw new IllegalArgumentException("query must contain searchable text");
02     }
03     return String.join(" AND ", phrases);
04 }
05 private static String sha256(byte[] content) {
06     try {
07         return HexFormat.of().formatHex(MessageDigest.getInstance("SHA-256").digest(content));
08     } catch (NoSuchAlgorithmException impossible) {
09         throw new IllegalStateException("SHA-256 is unavailable", impossible);
10     }
11 }
12 private static String decodeUtf8(byte[] content) {
13     try {
14         return StandardCharsets.UTF_8.newDecoder()
15             .onMalformedInput(CodingErrorAction.REPORT)
16             .onUnmappableCharacter(CodingErrorAction.REPORT)
17             .decode(ByteBuffer.wrap(content))
18             .toString();
19     } catch (CharacterCodingException error) {
20         throw new IllegalArgumentException("Knowledge document must use valid UTF-8", error);
21     }
22 }
23 private static String truncate(String value, int maxLength) {
24     return value.length() <= maxLength ? value : value.substring(0, maxLength);
25 }
26 private static String requireText(String value, String name) {
27     if (value == null || value.isBlank()) {
28         throw new IllegalArgumentException(name + " must not be blank");
29     }
30     return value.trim();
31 }
32 }
33 // FILE: src\main\java\com\sqlteacher\infrastructure\database\SqliteSchemaMigrator.java
34 package com.sqlteacher.infrastructure.database;
35 import java.nio.file.Path;
36 import java.sql.Connection;
37 import java.sql.DriverManager;
38 import java.sql.PreparedStatement;
39 import java.sql.ResultSet;
40 import java.sql.SQLException;
41 import java.sql.Statement;
42 import java.util.ArrayList;
43 import java.util.List;
44 final class SqliteSchemaMigrator {
45     private static final List<Migration> DEFAULT_MIGRATIONS = List.of(
46         new Migration(
47             1,
48             "Create the Demo baseline application tables",
49             List.of(
50                 ""
```

```
01         create table if not exists app_event (
02             id integer primary key autoincrement,
03             event_type text not null,
04             message text,
05             created_at text not null default current_timestamp
06         )
07         """
08     """
09     create table if not exists learning_events (
10         id integer primary key autoincrement,
11         event_type text not null,
12         occurred_at text not null,
13         connection_id text not null,
14         successful integer not null,
15         attributes text,
16         created_at text not null default current_timestamp
17     )
18     """
19 )
20 ),
21 new Migration(
22     2,
23     "Create database connection profile tables",
24     List.of(
25         """
26         create table connection_profiles (
27             id text primary key,
28             display_name text not null,
29             dialect text not null check (dialect in ('SQLITE', 'MYSQL', 'MARIADB')),
30             sqlite_path text,
31             host text,
32             port integer,
33             database_name text,
34             username text,
35             read_only integer not null check (read_only in (0, 1)),
36             enabled integer not null check (enabled in (0, 1)),
37             built_in integer not null check (built_in in (0, 1)),
38             created_at text not null default current_timestamp,
39             updated_at text not null default current_timestamp,
40             check (
41                 (dialect = 'SQLITE' and sqlite_path is not null
42                  and host is null and port is null and database_name is null and username is null)
43             or
44                 (dialect in ('MYSQL', 'MARIADB') and sqlite_path is null
45                  and host is not null and port between 1 and 65535
46                  and database_name is not null and username is not null)
47             )
48         )
49         """
50     """
```

```
01         create table connection_selection (
02             singleton_id integer primary key check (singleton_id = 1),
03             connection_id text not null,
04             updated_at text not null default current_timestamp
05         )
06         ""
07     )
08 ),
09 new Migration(
10     3,
11     "Create exercise catalog and attempt tables",
12     List.of(
13         ""
14         create table exercise_datasets (
15             id text primary key,
16             name text not null,
17             setup_sql text not null,
18             version integer not null check (version > 0),
19             created_at text not null default current_timestamp,
20             updated_at text not null default current_timestamp
21         )
22         "",
23         ""
24         create table exercises (
25             id text primary key,
26             title text not null,
27             description text not null,
28             knowledge_point text not null,
29             difficulty text not null check (
30                 difficulty in ('BEGINNER', 'INTERMEDIATE', 'ADVANCED')
31             ),
32             dataset_id text not null references exercise_datasets(id),
33             reference_sql text not null,
34             evaluation_rule_json text not null,
35             hints_json text not null,
36             version integer not null check (version > 0),
37             enabled integer not null check (enabled in (0, 1)),
38             created_at text not null default current_timestamp,
39             updated_at text not null default current_timestamp
40         )
41         "",
42         ""
43         create index exercises_enabled_order
44         on exercises(enabled, difficulty, knowledge_point, title)
45         "",
46         ""
47         create table exercise_sessions (
48             id text primary key,
49             exercise_id text not null references exercises(id),
50             exercise_version integer not null check (exercise_version > 0),
```

```
01         started_at text not null,
02         completed_at text,
03         hints_used integer not null default 0 check (hints_used between 0 and 3)
04     )
05     """
06 """
07     create index exercise_sessions_exercise_started
08     on exercise_sessions(exercise_id, started_at)
09     """
10 """
11     create table exercise_attempts (
12         id text primary key,
13         session_id text not null references exercise_sessions(id),
14         status text not null check (
15             status in ('RUN', 'SUBMITTED', 'PASSED', 'FAILED')
16         ),
17         sql_text text not null,
18         execution_success integer check (execution_success in (0, 1)),
19         passed integer check (passed in (0, 1)),
20         duration_ms integer not null check (duration_ms >= 0),
21         result_columns_json text not null default '[]',
22         result_rows_json text not null default '[]',
23         feedback_json text not null default '[]',
24         error_code text,
25         created_at text not null
26     )
27     """
28 """
29     create index exercise_attempts_session_created
30     on exercise_attempts(session_id, created_at)
31     """
32 )
33 ),
34 new Migration(
35     4,
36     "Create local knowledge document and FTS5 index tables",
37     List.of(
38         """
39         create table knowledge_documents (
40             id text primary key,
41             title text not null,
42             source_name text not null,
43             content_hash text not null unique,
44             chunk_count integer not null check (chunk_count > 0),
45             imported_at text not null
46         )
47         """
48 """
49         create table knowledge_chunks (
50             id text primary key,
```

```

01         document_id text not null references knowledge_documents(id) on delete cascade,
02         chunk_index integer not null check (chunk_index >= 0),
03         content text not null,
04         unique(document_id, chunk_index)
05     )
06     """
07 """
08     create virtual table knowledge_chunks_fts using fts5(
09         content,
10         content='knowledge_chunks',
11         content_rowid='rowid',
12         tokenize='unicode61 remove_diacritics 2'
13     )
14     """
15 """
16     create trigger knowledge_chunks_ai after insert on knowledge_chunks begin
17         insert into knowledge_chunks_fts(rowid, content) values (new.rowid, new.content);
18     end
19     """
20 """
21     create trigger knowledge_chunks_ad after delete on knowledge_chunks begin
22         insert into knowledge_chunks_fts(knowledge_chunks_fts, rowid, content)
23         values ('delete', old.rowid, old.content);
24     end
25     """
26 """
27     create trigger knowledge_chunks_au after update on knowledge_chunks begin
28         insert into knowledge_chunks_fts(knowledge_chunks_fts, rowid, content)
29         values ('delete', old.rowid, old.content);
30         insert into knowledge_chunks_fts(rowid, content) values (new.rowid, new.content);
31     end
32     """
33     "create index knowledge_chunks_document_order on knowledge_chunks(document_id, chunk_index)"
34 )
35 )
36 );
37 private final List<Migration> migrations;
38 SqliteSchemaMigrator() {
39     this(DEFAULT_MIGRATIONS);
40 }
41 SqliteSchemaMigrator(List<Migration> migrations) {
42     this.migrations = validateMigrations(migrations);
43 }
44 int migrate(Path databasePath) throws SQLException {
45     SqliteDriver.ensureLoaded();
46     try (Connection connection = DriverManager.getConnection("jdbc:sqlite:" + databasePath)) {
47         connection.setAutoCommit(false);
48         try {
49             createVersionTable(connection);
50             List<Integer> appliedVersions = readAppliedVersions(connection);

```

```
01         validateAppliedVersions(appliedVersions);
02         for (int index = appliedVersions.size(); index < migrations.size(); index++) {
03             applyMigration(connection, migrations.get(index));
04         }
05         connection.commit();
06         return latestVersion();
07     } catch (SQLException | RuntimeException error) {
08         rollback(connection, error);
09         throw error;
10     }
11 }
12 }
13 int latestVersion() {
14     return migrations.getLast().version();
15 }
16 int currentVersion(Path databasePath) throws SQLException {
17     if (java.nio.file.Files.notExists(databasePath)) {
18         return 0;
19     }
20     SqliteDriver.ensureLoaded();
21     try (Connection connection = DriverManager.getConnection("jdbc:sqlite:" + databasePath);
22          Statement statement = connection.createStatement()) {
23         try (ResultSet result = statement.executeQuery(
24             "select max(version) from schema_version"
25         )) {
26             return result.next() ? result.getInt(1) : 0;
27         } catch (SQLException missingVersionTable) {
28             return 0;
29         }
30     }
31 }
32 private static void createVersionTable(Connection connection) throws SQLException {
33     try (Statement statement = connection.createStatement()) {
34         statement.executeUpdate("""
35             create table if not exists schema_version (
36                 version integer primary key,
37                 description text not null,
38                 applied_at text not null default current_timestamp
39             )
40             """);
41     }
42 }
43 private static List<Integer> readAppliedVersions(Connection connection) throws SQLException {
44     List<Integer> versions = new ArrayList<>();
45     try (Statement statement = connection.createStatement();
46          ResultSet resultSet = statement.executeQuery("select version from schema_version order by version")) {
47         while (resultSet.next()) {
48             versions.add(resultSet.getInt("version"));
49         }
50     }
```

```
01     return List.copyOf(versions);
02 }
03 private void validateAppliedVersions(List<Integer> appliedVersions) throws SQLException {
04     if (appliedVersions.size() > migrations.size()) {
05         throw new SQLException("Application database schema is newer than this SQLTeacher version");
06     }
07     for (int index = 0; index < appliedVersions.size(); index++) {
08         int expected = migrations.get(index).version();
09         int actual = appliedVersions.get(index);
10         if (actual != expected) {
11             throw new SQLException(
12                 "Application database migration history is invalid: expected version "
13                 + expected + " but found " + actual
14             );
15         }
16     }
17 }
18 private static void applyMigration(Connection connection, Migration migration) throws SQLException {
19     for (String sql : migration.statements()) {
20         try (Statement statement = connection.createStatement()) {
21             statement.executeUpdate(sql);
22         }
23     }
24     try (PreparedStatement statement = connection.prepareStatement(
25         "insert into schema_version(version, description) values (?, ?)"
26     )) {
27         statement.setInt(1, migration.version());
28         statement.setString(2, migration.description());
29         statement.executeUpdate();
30     }
31 }
32 private static void rollback(Connection connection, Throwable originalError) {
33     try {
34         connection.rollback();
35     } catch (SQLException rollbackError) {
36         originalError.addSuppressed(rollbackError);
37     }
38 }
39 private static List<Migration> validateMigrations(List<Migration> migrations) {
40     if (migrations == null || migrations.isEmpty()) {
41         throw new IllegalArgumentException("At least one SQLite schema migration is required");
42     }
43     List<Migration> copy = List.copyOf(migrations);
44     for (int index = 0; index < copy.size(); index++) {
45         int expectedVersion = index + 1;
46         if (copy.get(index).version() != expectedVersion) {
47             throw new IllegalArgumentException(
48                 "SQLite schema migrations must be ordered and contiguous from version 1"
49             );
50         }
51     }
52 }
```

```
01     }
02     return copy;
03 }
04 record Migration(int version, String description, List<String> statements) {
05     Migration {
06         if (version < 1) {
07             throw new IllegalArgumentException("Migration version must be positive");
08         }
09         if (description == null || description.isBlank()) {
10             throw new IllegalArgumentException("Migration description must not be blank");
11         }
12         if (statements == null || statements.isEmpty()) {
13             throw new IllegalArgumentException("Migration statements must not be empty");
14         }
15         statements = List.copyOf(statements);
16         if (statements.stream().anyMatch(sql -> sql == null || sql.isBlank())) {
17             throw new IllegalArgumentException("Migration SQL must not be blank");
18         }
19     }
20 }
21 }
22 // FILE: src\main\java\com\sqlteacher\infrastructure\database\SqlResultMapper.java
23 package com.sqlteacher.infrastructure.database;
24 import com.sqlteacher.application.execution.SqlExecutionResult;
25 import java.sql.ResultSet;
26 import java.sql.ResultSetMetaData;
27 import java.sql.SQLException;
28 import java.time.Duration;
29 import java.util.ArrayList;
30 import java.util.LinkedHashMap;
31 import java.util.List;
32 import java.util.Map;
33 public final class SqlResultMapper {
34     public SqlExecutionResult mapQueryResult(
35         ResultSet resultSet,
36         Duration duration,
37         int maxRows
38     ) throws SQLException {
39         List<String> columns = new ArrayList<>();
40         List<Map<String, Object>> rows = new ArrayList<>();
41         ResultSetMetaData metaData = resultSet.getMetaData();
42         int columnCount = metaData.getColumnCount();
43         for (int i = 1; i <= columnCount; i++) {
44             columns.add(metaData.getColumnLabel(i));
45         }
46         boolean truncated = false;
47         while (resultSet.next()) {
48             if (rows.size() >= maxRows) {
49                 truncated = true;
50                 break;
```

```
01         }
02         Map<String, Object> row = new LinkedHashMap<>();
03         for (int i = 1; i <= columnCount; i++) {
04             row.put(
05                 metaData.getColumnLabel(i),
06                 resultSet.getObject(i)
07             );
08         }
09         rows.add(row);
10     }
11     return new SqlExecutionResult(
12         true,
13         columns,
14         rows,
15         0,
16         truncated,
17         "SQL executed successfully",
18         duration
19     );
20 }
21 public SqlExecutionResult mapUpdateResult(
22     int affectedRows,
23     Duration duration
24 ) {
25     return new SqlExecutionResult(
26         true,
27         List.of(),
28         List.of(),
29         affectedRows,
30         false,
31         "SQL executed successfully",
32         duration
33     );
34 }
35 }
36 // FILE: src\main\java\com\sqlteacher\infrastructure\database\SqlScriptSplitter.java
37 package com.sqlteacher.infrastructure.database;
38 import java.util.ArrayList;
39 import java.util.List;
40 final class SqlScriptSplitter {
41     private SqlScriptSplitter() {
42     }
43     static List<String> split(String script) {
44         List<String> statements = new ArrayList<>();
45         StringBuilder current = new StringBuilder();
46         char quote = '\0';
47         boolean lineComment = false;
48         boolean blockComment = false;
49         for (int index = 0; index < script.length(); index++) {
50             char character = script.charAt(index);
```

```
01      char next = index + 1 < script.length() ? script.charAt(index + 1) : '\0';
02      if (lineComment) {
03          current.append(character);
04          if (character == '\n' || character == '\r') {
05              lineComment = false;
06          }
07          continue;
08      }
09      if (blockComment) {
10          current.append(character);
11          if (character == '*' && next == '/') {
12              current.append(next);
13              index++;
14              blockComment = false;
15          }
16          continue;
17      }
18      if (quote == '\0' && character == '-' && next == '-') {
19          current.append(character).append(next);
20          index++;
21          lineComment = true;
22          continue;
23      }
24      if (quote == '\0' && character == '/' && next == '*') {
25          current.append(character).append(next);
26          index++;
27          blockComment = true;
28          continue;
29      }
30      if (quote == '\0' && (character == '\'' || character == '"' || character == '`')) {
31          quote = character;
32          current.append(character);
33          continue;
34      }
35      if (quote != '\0' && character == quote) {
36          current.append(character);
37          if (next == quote) {
38              current.append(next);
39              index++;
40          } else {
41              quote = '\0';
42          }
43          continue;
44      }
45      if (quote == '\0' && character == ';') {
46          addStatement(statements, current);
47      } else {
48          current.append(character);
49      }
50  }
```

```
01         addStatement(statements, current);
02         return List.copyOf(statements);
03     }
04     private static void addStatement(List<String> statements, StringBuilder current) {
05         String statement = current.toString().trim();
06         if (!statement.isEmpty()) {
07             statements.add(statement);
08         }
09         current.setLength(0);
10     }
11 }
12 // FILE: src\main\java\com\sqlteacher\infrastructure\environment\JavaFxEnvironmentVerifier.java
13 package com.sqlteacher.infrastructure.environment;
14 import java.awt.GraphicsEnvironment;
15 public final class JavaFxEnvironmentVerifier {
16     public VerificationItem verifyRuntime() {
17         try {
18             Class.forName("javafx.application.Application");
19             Class.forName("javafx.stage.Stage");
20             return VerificationItem.passed("JavaFX runtime", "required JavaFX classes are available");
21         } catch (ClassNotFoundException ex) {
22             return VerificationItem.failed("JavaFX runtime", "required JavaFX classes are missing");
23         }
24     }
25     public VerificationItem verifyGraphicsEnvironment() {
26         if (GraphicsEnvironment.isHeadless()) {
27             return VerificationItem.warning(
28                 "Graphics environment",
29                 "headless environment detected; use CLI verification and skip JavaFX window launch"
30             );
31         }
32         return VerificationItem.passed(
33             "Graphics environment",
34             "desktop graphics environment is available for JavaFX manual verification"
35         );
36     }
37 }
38 // FILE: src\main\java\com\sqlteacher\infrastructure\environment\RuntimeEnvironment.java
39 package com.sqlteacher.infrastructure.environment;
40 public record RuntimeEnvironment(String javaVersion, String osName, String osVersion, String architecture) {
41     public static RuntimeEnvironment detect() {
42         return new RuntimeEnvironment(
43             System.getProperty("java.version"),
44             System.getProperty("os.name"),
45             System.getProperty("os.version"),
46             System.getProperty("os.arch")
47         );
48     }
49     public VerificationItem javaVersionItem() {
50         return VerificationItem.passed("Java runtime", javaVersion);
```

```
01     }
02     public VerificationItem osItem() {
03         return VerificationItem.passed("Operating system", osName + " " + osVersion + " (" + architecture + ")");
04     }
05 }
06 // FILE: src\main\java\com\sqlteacher\infrastructure\environment\VerificationItem.java
07 package com.sqlteacher.infrastructure.environment;
08 public record VerificationItem(String name, VerificationStatus status, String detail) {
09     public static VerificationItem passed(String name, String detail) {
10         return new VerificationItem(name, VerificationStatus.PASS, detail);
11     }
12     public static VerificationItem warning(String name, String detail) {
13         return new VerificationItem(name, VerificationStatus.WARNING, detail);
14     }
15     public static VerificationItem failed(String name, String detail) {
16         return new VerificationItem(name, VerificationStatus.FAIL, detail);
17     }
18 }
19 // FILE: src\main\java\com\sqlteacher\infrastructure\environment\VerificationStatus.java
20 package com.sqlteacher.infrastructure.environment;
21 public enum VerificationStatus {
22     PASS,
23     WARNING,
24     FAIL
25 }
26 // FILE: src\main\java\com\sqlteacher\infrastructure\spring\MockApplicationServiceConfig.java
27 package com.sqlteacher.infrastructure.spring;
28 import com.sqlteacher.application.ai.AiStatusService;
29 import com.sqlteacher.application.error.ApplicationExceptionMapper;
30 import com.sqlteacher.application.error.DefaultApplicationExceptionMapper;
31 import com.sqlteacher.application.execution.SqlExecutionService;
32 import com.sqlteacher.application.event.LearningEventService;
33 import com.sqlteacher.application.metadata.DatabaseMetadataService;
34 import com.sqlteacher.application.mock.MockAiStatusService;
35 import com.sqlteacher.application.mock.MockDatabaseMetadataService;
36 import com.sqlteacher.application.mock.MockNl2SqlService;
37 import com.sqlteacher.application.mock.MockLearningEventService;
38 import com.sqlteacher.application.mock.MockSqlExecutionService;
39 import com.sqlteacher.application.mock.MockSqlRiskAnalysisService;
40 import com.sqlteacher.application.nl2sql.Nl2SqlService;
41 import com.sqlteacher.application.nl2sql.DefaultNl2SqlSafetyService;
42 import com.sqlteacher.application.nl2sql.Nl2SqlSafetyService;
43 import com.sqlteacher.application.risk.SqlRiskAnalysisService;
44 import org.springframework.context.annotation.Bean;
45 import org.springframework.context.annotation.Configuration;
46 @Configuration
47 public class MockApplicationServiceConfig {
48     @Bean
49     public SqlRiskAnalysisService sqlRiskAnalysisService() {
50         return new MockSqlRiskAnalysisService();
```

```
01     }
02     @Bean
03     public SqlExecutionService sqlExecutionService(SqlRiskAnalysisService sqlRiskAnalysisService) {
04         return new MockSqlExecutionService(sqlRiskAnalysisService);
05     }
06     @Bean
07     public DatabaseMetadataService databaseMetadataService() {
08         return new MockDatabaseMetadataService();
09     }
10     @Bean
11     public Nl2SqlService nl2SqlService() {
12         return new MockNl2SqlService();
13     }
14     @Bean
15     public LearningEventService learningEventService() {
16         return new MockLearningEventService();
17     }
18     @Bean
19     public Nl2SqlSafetyService nl2SqlSafetyService(
20         Nl2SqlService nl2SqlService,
21         SqlRiskAnalysisService sqlRiskAnalysisService,
22         LearningEventService learningEventService
23     ) {
24         return new DefaultNl2SqlSafetyService(
25             nl2SqlService,
26             sqlRiskAnalysisService,
27             learningEventService
28         );
29     }
30     @Bean
31     public AiStatusService mockAiStatusService() {
32         return new MockAiStatusService();
33     }
34     @Bean
35     public ApplicationExceptionMapper applicationExceptionHandler() {
36         return new DefaultApplicationExceptionHandler();
37     }
38 }
39 // FILE: src\main\java\com\sqlteacher\infrastructure\spring\SqlTeacherApplicationConfig.java
40 package com.sqlteacher.infrastructure.spring;
41 import com.sqlteacher.application.ai.AiModelProvider;
42 import com.sqlteacher.application.ai.AiModelSelectionService;
43 import com.sqlteacher.application.ai.AiStatusService;
44 import com.sqlteacher.application.ai.NetworkAiSettingsService;
45 import com.sqlteacher.application.config.AppConfigurationService;
46 import com.sqlteacher.application.config.SqlTeacherConfiguration;
47 import com.sqlteacher.application.collaboration.CloudApiClient;
48 import com.sqlteacher.application.collaboration.CloudSessionService;
49 import com.sqlteacher.application.collaboration.CloudLearningSyncService;
50 import com.sqlteacher.application.event.LearningEventQueryService;
```

```
01 import com.sqlteacher.application.event.LearningEventRecorder;
02 import com.sqlteacher.application.connection.ConnectionManagementService;
03 import com.sqlteacher.application.database.DatabaseInitializationService;
04 import com.sqlteacher.application.error.ApplicationExceptionMapper;
05 import com.sqlteacher.application.error.DefaultApplicationExceptionMapper;
06 import com.sqlteacher.application.event.LearningEventService;
07 import com.sqlteacher.application.metadata.DatabaseMetadataService;
08 import com.sqlteacher.application.nl2sql.DefaultNl2SqlSafetyService;
09 import com.sqlteacher.application.nl2sql.Nl2SqlService;
10 import com.sqlteacher.application.nl2sql.Nl2SqlSafetyService;
11 import com.sqlteacher.application.risk.SqlRiskAnalysisService;
12 import com.sqlteacher.infrastructure.ai.Nl2SqlServiceImpl;
13 import com.sqlteacher.infrastructure.ai.OllamaAiModelProvider;
14 import com.sqlteacher.infrastructure.ai.OllamaAiStatusService;
15 import com.sqlteacher.infrastructure.ai.OllamaModelSelectionService;
16 import com.sqlteacher.infrastructure.ai.InMemoryNetworkAiSettingsService;
17 import com.sqlteacher.infrastructure.ai.SwitchableAiModelProvider;
18 import com.sqlteacher.infrastructure.config.PropertiesAppConfigurationService;
19 import com.sqlteacher.infrastructure.cloud.HttpCloudApiClient;
20 import com.sqlteacher.infrastructure.cloud.InMemoryCloudSessionService;
21 import com.sqlteacher.infrastructure.cloud.DefaultCloudLearningSyncService;
22 import com.sqlteacher.infrastructure.database.DatabaseServiceConfig;
23 import com.sqlteacher.infrastructure.database.SQLiteAppDatabaseInitializer;
24 import org.springframework.context.annotation.Bean;
25 import org.springframework.context.annotation.Configuration;
26 import org.springframework.context.annotation.Import;
27 import java.net.URI;
28 @Configuration
29 @Import(DatabaseServiceConfig.class)
30 public class SqlTeacherApplicationConfig {
31     @Bean
32     public AppConfigurationService appConfigurationService() {
33         return new PropertiesAppConfigurationService();
34     }
35     @Bean
36     public SqlTeacherConfiguration sqlTeacherConfiguration(AppConfigurationService appConfigurationService) {
37         return appConfigurationService.current();
38     }
39     @Bean
40     public DatabaseInitializationService databaseInitializationService(SqlTeacherConfiguration properties) {
41         return new SQLiteAppDatabaseInitializer(properties);
42     }
43     @Bean
44     public AiStatusService aiStatusService(SqlTeacherConfiguration properties) {
45         return new OllamaAiStatusService(properties.ai());
46     }
47     @Bean
48     public AiModelProvider aiModelProvider(SqlTeacherConfiguration properties, AiStatusService aiStatusService,
49         NetworkAiSettingsService networkSettings) {
50         return new SwitchableAiModelProvider(new OllamaAiModelProvider(properties.ai(), aiStatusService),
```

```
01     > networkSettings);
02 }
03 @Bean(destroyMethod = "clear")
04 public NetworkAiSettingsService networkAiSettingsService() { return new InMemoryNetworkAiSettingsService(); }
05 @Bean
06 public AiModelSelectionService aiModelSelectionService(SqlTeacherConfiguration properties) {
07     return new OllamaModelSelectionService(
08         properties.ai(),
09         properties.dataDirectory().resolve("selected-ai-model.txt")
10     );
11 }
12 @Bean
13 public Nl2SqlService nl2SqlService(
14     AiModelProvider aiModelProvider,
15     SqlTeacherConfiguration properties,
16     AiModelSelectionService modelSelectionService,
17     DatabaseMetadataService databaseMetadataService,
18     LearningEventService learningEventService,
19     ConnectionManagementService connectionManagementService
20 ) {
21     return new Nl2SqlServiceImpl(
22         aiModelProvider,
23         properties.ai(),
24         modelSelectionService,
25         databaseMetadataService,
26         learningEventService,
27         connectionManagementService
28     );
29 }
30 @Bean
31 public Nl2SqlSafetyService nl2SqlSafetyService(
32     Nl2SqlService nl2SqlService,
33     SqlRiskAnalysisService riskAnalysisService,
34     LearningEventService learningEventService
35 ) {
36     return new DefaultNl2SqlSafetyService(
37         nl2SqlService,
38         riskAnalysisService,
39         learningEventService
40     );
41 }
42 @Bean
43 public ApplicationExceptionHandler applicationExceptionHandler() {
44     return new DefaultApplicationExceptionHandler();
45 }
46 @Bean
47 public CloudApiClient cloudApiClient() {
48     return new HttpCloudApiClient(URI.create(System.getProperty("sqlteacher.cloud.base-url",
49 > "http://8.130.47.235"))));
50 }
```

```
01     @Bean
02     public CloudSessionService cloudSessionService() {
03         return new InMemoryCloudSessionService();
04     }
05     @Bean
06     public CloudLearningSyncService cloudLearningSyncService(CloudApiClient api, CloudSessionService sessions,
07         LearningEventQueryService query, LearningEventRecorder recorder, SqlTeacherConfiguration configuration) {
08         return new DefaultCloudLearningSyncService(api, sessions, query, recorder,
09             configuration.dataDirectory().resolve("cloud-state"));
10     }
11 }
12 // FILE: src\main\java\com\sqlteacher\ReleaseVerificationApp.java
13 package com.sqlteacher;
14 import com.sqlteacher.application.analytics.AnalyticsFilter;
15 import com.sqlteacher.application.analytics.LearningAnalyticsService;
16 import com.sqlteacher.application.config.SqlTeacherConfiguration;
17 import com.sqlteacher.application.database.DatabaseInitializationService;
18 import com.sqlteacher.application.execution.SqlExecutionRequest;
19 import com.sqlteacher.application.execution.SqlExecutionService;
20 import com.sqlteacher.application.knowledge.KnowledgeDocumentService;
21 import com.sqlteacher.application.knowledge.KnowledgeSearchService;
22 import com.sqlteacher.application.metadata.DatabaseMetadataService;
23 import com.sqlteacher.infrastructure.spring.SqlTeacherApplicationConfig;
24 import org.springframework.context.annotation.AnnotationConfigApplicationContext;
25 import java.nio.file.Files;
26 import java.nio.file.Path;
27 import java.sql.DriverManager;
28 import java.time.Duration;
29 import java.util.Comparator;
30 public final class ReleaseVerificationApp {
31     private static final long STARTUP_LIMIT_MS = 5_000;
32     private static final long OPERATION_LIMIT_MS = 2_000;
33     private ReleaseVerificationApp() {
34     }
35     public static void main(String[] args) throws Exception {
36         Path workspace = Files.createTempDirectory("sqlteacher-release-verification-");
37         String previousDataDirectory = System.getProperty("sqlteacher.data.dir");
38         System.setProperty("sqlteacher.data.dir", workspace.toString());
39         try {
40             long startupStarted = System.nanoTime();
41             try (AnnotationConfigApplicationContext context =
42                 new AnnotationConfigApplicationContext(SqlTeacherApplicationConfig.class)) {
43                 context.getBean(DatabaseInitializationService.class).initialize();
44                 long startupMs = elapsedMillis(startupStarted);
45                 SqlTeacherConfiguration configuration = context.getBean(SqlTeacherConfiguration.class);
46                 populateFiveHundredRows(configuration.database().demoDatabasePath());
47                 long metadataStarted = System.nanoTime();
48                 int tableCount = context.getBean(DatabaseMetadataService.class).listTables("demo").size();
49                 long metadataMs = elapsedMillis(metadataStarted);
50                 long queryStarted = System.nanoTime();
```

```
01         var query = context.getBean(SqlExecutionService.class).execute(
02             new SqlExecutionRequest("demo", "select id, name, score from student order by id", 500,
03                 Duration.ofSeconds(5))
04         );
05         long queryMs = elapsedMillis(queryStarted);
06         long analyticsStarted = System.nanoTime();
07         context.getBean(LearningAnalyticsService.class).analyze(AnalyticsFilter.all());
08         long analyticsMs = elapsedMillis(analyticsStarted);
09         Path knowledgeFile = workspace.resolve("release-verification.md");
10         Files.writeString(knowledgeFile, "# SQL ■■■\nGROUP BY ■■■■■■HAVING ■■■■■■■■■■\n");
11         KnowledgeDocumentService documents = context.getBean(KnowledgeDocumentService.class);
12         KnowledgeSearchService search = context.getBean(KnowledgeSearchService.class);
13         documents.importDocument(knowledgeFile);
14         long searchStarted = System.nanoTime();
15         int searchResults = search.search("GROUP BY", 5).size();
16         long searchMs = elapsedMillis(searchStarted);
17         check("startup", startupMs, STARTUP_LIMIT_MS, true);
18         check("metadata", metadataMs, OPERATION_LIMIT_MS, tableCount > 0);
19         check("500-row query", queryMs, OPERATION_LIMIT_MS, query.success() && query.rows().size() == 500);
20         check("analytics", analyticsMs, OPERATION_LIMIT_MS, true);
21         check("local search", searchMs, OPERATION_LIMIT_MS, searchResults > 0);
22     }
23 } finally {
24     if (previousDataDirectory == null) {
25         System.clearProperty("sqlteacher.data.dir");
26     } else {
27         System.setProperty("sqlteacher.data.dir", previousDataDirectory);
28     }
29     deleteTree(workspace);
30 }
31 }
32 private static void populateFiveHundredRows(Path database) throws Exception {
33     try (var connection = DriverManager.getConnection("jdbc:sqlite:" + database)) {
34         connection.setAutoCommit(false);
35         try (var statement = connection.prepareStatement(
36             "insert into student(id, name, score) values (?, ?, ?)")) {
37             for (int id = 3; id <= 500; id++) {
38                 statement.setInt(1, id);
39                 statement.setString(2, "Student " + id);
40                 statement.setInt(3, id % 101);
41                 statement.addBatch();
42             }
43             statement.executeBatch();
44         }
45         connection.commit();
46     }
47 }
48 private static void check(String name, long elapsedMs, long limitMs, boolean correct) {
49     boolean passed = correct && elapsedMs <= limitMs;
50     System.out.printf("[%s] %s - %d ms (limit %d ms)%n", passed ? "PASS" : "FAIL", name, elapsedMs, limitMs);
```

```
01         if (!passed) {
02             throw new IllegalStateException("Release performance verification failed: " + name);
03         }
04     }
05     private static long elapsedMillis(long startedNanos) {
06         return Duration.ofNanos(System.nanoTime() - startedNanos).toMillis();
07     }
08     private static void deleteTree(Path root) {
09         if (root == null || Files.notExists(root)) {
10             return;
11         }
12         try (var paths = Files.walk(root)) {
13             paths.sorted(Comparator.reverseOrder()).forEach(path -> {
14                 try {
15                     Files.deleteIfExists(path);
16                 } catch (Exception ignored) {
17                     // Temporary release verification data is best-effort cleanup only.
18                 }
19             });
20         } catch (Exception ignored) {
21             // Temporary release verification data is best-effort cleanup only.
22         }
23     }
24 }
25 // FILE: src\main\java\com\sqlteacher\server\SqlTeacherCloudServer.java
26 package com.sqlteacher.server;
27 import com.fasterxml.jackson.core.type.TypeReference;
28 import com.fasterxml.jackson.databind.ObjectMapper;
29 import com.sqlteacher.application.collaboration.AuthenticatedUser;
30 import com.sqlteacher.application.collaboration.ClassroomService;
31 import com.sqlteacher.application.collaboration.CloudAuthenticationService;
32 import com.sqlteacher.application.collaboration.CloudSyncItem;
33 import com.sqlteacher.application.collaboration.ClassAssignment;
34 import com.sqlteacher.application.collaboration.UserRole;
35 import com.sun.net.httpserver.HttpExchange;
36 import com.sun.net.httpserver.HttpServer;
37 import java.io.IOException;
38 import java.net.InetSocketAddress;
39 import java.nio.charset.StandardCharsets;
40 import java.nio.file.Files;
41 import java.nio.file.Path;
42 import java.security.GeneralSecurityException;
43 import java.security.SecureRandom;
44 import java.security.spec.KeySpec;
45 import java.sql.Connection;
46 import java.sql.DriverManager;
47 import java.sql.PreparedStatement;
48 import java.sql.ResultSet;
49 import java.sql.SQLException;
50 import java.sql.Statement;
```

```
01 import java.time.Instant;
02 import java.time.temporal.ChronoUnit;
03 import java.util.ArrayList;
04 import java.util.Base64;
05 import java.util.LinkedHashMap;
06 import java.util.List;
07 import java.util.Locale;
08 import java.util.Map;
09 import java.util.Set;
10 import java.util.UUID;
11 import java.util.concurrent.Executors;
12 import javax.crypto.SecretKeyFactory;
13 import javax.crypto.spec.PBEKeySpec;
14 /**
15  * Small deployable cloud API for v1.1. It intentionally exposes only account and class APIs;
16  * desktop database credentials and BYO-AI keys never cross this boundary.
17  */
18 public final class SqlTeacherCloudServer {
19     private static final ObjectMapper JSON = new ObjectMapper().findAndRegisterModules();
20     private static final int TOKEN_BYTES = 32;
21     private static final int SALT_BYTES = 16;
22     private static final int PBKDF2_ITERATIONS = 310_000;
23     private static final int HASH_BITS = 256;
24     private final CloudStore store;
25     private final HttpServer server;
26     private SqlTeacherCloudServer(Path databasePath, int port) throws IOException, SQLException {
27         this.store = new CloudStore(databasePath);
28         String bootstrapEmail = System.getenv("SQLTEACHER_CLOUD_BOOTSTRAP_ADMIN_EMAIL");
29         String bootstrapPassword = System.getenv("SQLTEACHER_CLOUD_BOOTSTRAP_ADMIN_PASSWORD");
30         if (bootstrapEmail != null && !bootstrapEmail.isBlank()
31             && bootstrapPassword != null && !bootstrapPassword.isBlank()) {
32             store.ensureBootstrapAdmin(bootstrapEmail, bootstrapPassword.toCharArray());
33         }
34         this.server = HttpServer.create(new InetSocketAddress("127.0.0.1", port), 0);
35         this.server.createContext("/health", this::health);
36         this.server.createContext("/api/v1/auth/register", this::register);
37         this.server.createContext("/api/v1/auth/login", this::login);
38         this.server.createContext("/api/v1/auth/logout", this::logout);
39         this.server.createContext("/api/v1/classes", this::classes);
40         this.server.createContext("/api/v1/sync/events", this::syncEvents);
41         this.server.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
42     }
43     public static void main(String[] args) throws Exception {
44         int port = Integer.parseInt(System.getenv().getOrDefault("SQLTEACHER_CLOUD_PORT", "8080"));
45         Path database = Path.of(System.getenv().getOrDefault("SQLTEACHER_CLOUD_DB", "./data/cloud.db"))
46             .toAbsolutePath().normalize();
47         SqlTeacherCloudServer cloudServer = new SqlTeacherCloudServer(database, port);
48         cloudServer.server.start();
49         System.out.println("SQLTeacher cloud API started on port " + port);
50     }
```

```
01     private void health(HttpExchange exchange) throws IOException {
02         if (!"GET".equals(exchange.getRequestMethod())) { methodNotAllowed(exchange); return; }
03         respond(exchange, 200, Map.of("status", "ok", "time", Instant.now().toString()));
04     }
05     private void register(HttpExchange exchange) throws IOException {
06         if (!"POST".equals(exchange.getRequestMethod())) { methodNotAllowed(exchange); return; }
07         try {
08             Map<String, String> body = request(exchange);
09             SessionData session = store.registerData(body.get("email"), body.get("displayName"), password(body));
10             respond(exchange, 201, sessionResponse(session));
11         } catch (IllegalArgumentException error) { respond(exchange, 400, errorResponse("INVALID_REQUEST",
12 > error.getMessage())); }
13         catch (SecurityException error) { respond(exchange, 409, errorResponse("ACCOUNT_EXISTS", "This email is
14 > already registered.")); }
15         catch (RuntimeException error) { error.printStackTrace(); respond(exchange, 500, errorResponse("SERVER_ERROR",
16 > "Registration failed.")); }
17         finally { clearPassword(exchange); }
18     }
19     private void login(HttpExchange exchange) throws IOException {
20         if (!"POST".equals(exchange.getRequestMethod())) { methodNotAllowed(exchange); return; }
21         try {
22             Map<String, String> body = request(exchange);
23             SessionData session = store.loginData(body.get("email"), password(body));
24             respond(exchange, 200, sessionResponse(session));
25         } catch (IllegalArgumentException error) { respond(exchange, 400, errorResponse("INVALID_REQUEST",
26 > error.getMessage())); }
27         catch (SecurityException error) { respond(exchange, 401, errorResponse("LOGIN_FAILED", "Email or password is
28 > incorrect.")); }
29         catch (RuntimeException error) { error.printStackTrace(); respond(exchange, 500, errorResponse("SERVER_ERROR",
30 > "Login failed.")); }
31         finally { clearPassword(exchange); }
32     }
33     private void logout(HttpExchange exchange) throws IOException {
34         if (!"POST".equals(exchange.getRequestMethod())) { methodNotAllowed(exchange); return; }
35         try {
36             store.logout(token(exchange));
37             exchange.sendResponseHeaders(204, -1);
38         } catch (SecurityException error) { respond(exchange, 401, errorResponse("UNAUTHORIZED", "Login is
39 > required.")); }
40     }
41     private void classes(HttpExchange exchange) throws IOException {
42         try {
43             AuthenticatedUser actor = store.authenticate(token(exchange));
44             String path = exchange.getRequestURI().getPath();
45             if ("/api/v1/classes".equals(path) && "GET".equals(exchange.getRequestMethod())) {
46                 respond(exchange, 200, Map.of("classes", store.listVisibleTo(actor)));
47                 return;
48             }
49             if ("/api/v1/classes".equals(path) && "POST".equals(exchange.getRequestMethod())) {
50                 Map<String, String> body = request(exchange);
```

```
01         respond(exchange, 201, store.create(actor, body.get("name")));
02     return;
03 }
04 String[] segments = path.split("/");
05 if (segments.length == 6 && "members".equals(segments[5]) && "POST".equals(exchange.getRequestMethod())) {
06     Map<String, String> body = request(exchange);
07     UserRole role = UserRole.valueOf(body.get("role").toUpperCase(Locale.ROOT));
08     String userId = body.get("userId");
09     if ((userId == null || userId.isBlank()) && body.get("email") != null) userId =
10 > store.userIdByEmail(body.get("email"));
11     respond(exchange, 200, store.addMember(actor, segments[4], userId, role));
12     return;
13 }
14 if (segments.length == 6 && "assignments".equals(segments[5])) {
15     if ("GET".equals(exchange.getRequestMethod())) {
16         respond(exchange, 200, Map.of("assignments", store.listAssignments(actor, segments[4])); return;
17     }
18     if ("POST".equals(exchange.getRequestMethod())) {
19         Map<String, String> body = request(exchange);
20
21 > respond(exchange, 201, store.createAssignment(actor, segments[4], body.get("exerciseId"), body.get("title"))); return;
22     }
23 }
24 respond(exchange, 404, ErrorResponse("NOT_FOUND", "API endpoint was not found.));
25 } catch (SecurityException error) { respond(exchange, 403, ErrorResponse("FORBIDDEN", "You do not have access
26 > to this resource.)); }
27 catch (IllegalArgumentException error) { respond(exchange, 400, ErrorResponse("INVALID_REQUEST",
28 > error.getMessage())); }
29 catch (RuntimeException error) { error.printStackTrace(); respond(exchange, 500, ErrorResponse("SERVER_ERROR",
30 > "Classroom operation failed.)); }
31 }
32 private void syncEvents(HttpExchange exchange) throws IOException {
33     try {
34         AuthenticatedUser actor = store.authenticate(token(exchange));
35         if ("POST".equals(exchange.getRequestMethod())) {
36             SyncUpload upload = JSON.readValue(exchange.getRequestBody(), SyncUpload.class);
37             respond(exchange, 200, Map.of("accepted", store.upload(actor, upload.items())));
38             return;
39         }
40         if ("GET".equals(exchange.getRequestMethod())) {
41             long after = queryLong(exchange.getRequestURI().getRawQuery(), "afterVersion", 0);
42             respond(exchange, 200, Map.of("items", store.download(actor, after));
43             return;
44         }
45         methodNotAllowed(exchange);
46     } catch (SecurityException error) {
47         respond(exchange, 401, ErrorResponse("UNAUTHORIZED", "Login is required.));
48     } catch (IllegalArgumentException error) {
49         respond(exchange, 400, ErrorResponse("INVALID_REQUEST", error.getMessage()));
50     } catch (RuntimeException error) {
```

```
01         error.printStackTrace();
02         respond(exchange, 500, errorResponse("SERVER_ERROR", "Synchronization failed.));
03     }
04 }
05 private static long queryLong(String query, String name, long defaultValue) {
06     if (query == null || query.isBlank()) return defaultValue;
07     for (String pair : query.split("&")) {
08         String[] parts = pair.split("=", 2);
09         if (parts.length == 2 && name.equals(parts[0])) return Long.parseLong(parts[1]);
10     }
11     return defaultValue;
12 }
13 private static Map<String, String> request(HttpExchange exchange) throws IOException {
14     Map<String, Object> decoded = JSON.readValue(exchange.getRequestBody(), new TypeReference<>() { });
15     Map<String, String> result = new LinkedHashMap<>();
16     decoded.forEach((key, value) -> result.put(key, value == null ? "" : String.valueOf(value)));
17     return result;
18 }
19 private static char[] password(Map<String, String> body) { return body.getDefault("password", "").toCharArray();
20 > }
21 private static void clearPassword(HttpExchange ignored) { /* request body is released after this handler returns
22 > */ }
23 private static String token(HttpExchange exchange) {
24     String header = exchange.getRequestHeaders().getFirst("Authorization");
25     if (header == null || !header.startsWith("Bearer ") || header.length() < 20) throw new
26 > SecurityException("Missing bearer token");
27     return header.substring("Bearer ".length());
28 }
29 private static void methodNotAllowed(HttpExchange exchange) throws IOException {
30     exchange.getResponseHeaders().set("Allow", "GET, POST");
31     respond(exchange, 405, errorResponse("METHOD_NOT_ALLOWED", "Method not allowed.));
32 }
33 private static void respond(HttpExchange exchange, int status, Object body) throws IOException {
34     byte[] bytes = JSON.writeValueAsBytes(body);
35     exchange.getResponseHeaders().set("Content-Type", "application/json; charset=utf-8");
36     exchange.getResponseHeaders().set("Cache-Control", "no-store");
37     exchange.sendResponseHeaders(status, bytes.length);
38     exchange.getResponseBody().write(bytes);
39     exchange.close();
40 }
41 private static Map<String, Object> errorResponse(String code, String message) { return Map.of("code", code,
42 > "message", message); }
43 private static Map<String, Object> sessionResponse(SessionData session) {
44     return Map.of("accessToken", session.token(), "expiresAt", session.expiresAt().toString(), "user",
45 > session.user());
46 }
47 private record SyncUpload(List<CloudSyncItem> items) {
48     private SyncUpload { items = items == null ? List.of() : List.copyOf(items); }
49 }
50 private static final class CloudStore implements CloudAuthenticationService, ClassroomService {
```

```
01     private final Path database;
02     private final SecureRandom random = new SecureRandom();
03     private CloudStore(Path database) throws SQLException, IOException {
04         this.database = database;
05         Files.createDirectories(database.getParent());
06         try {
07             Class.forName("org.sqlite.JDBC");
08             initialize();
09         } catch (ClassNotFoundException error) {
10             throw new SQLException("SQLite JDBC driver is unavailable", error);
11         }
12     }
13     @Override public Session register(String email, String displayName, char[] password) { return
14 > toSession(registerData(email, displayName, password)); }
15     @Override public Session login(String email, char[] password) { return toSession(loginData(email, password));
16 > }
17     @Override public AuthenticatedUser authenticate(String accessToken) { return authenticateData(accessToken); }
18     @Override public void logout(String accessToken) { revoke(accessToken); }
19     SessionData registerData(String email, String displayName, char[] password) {
20         String normalizedEmail = validateEmail(email);
21         if (displayName == null || displayName.isBlank() || displayName.length() > 80) throw new
22 > IllegalArgumentException("displayName must be 1 to 80 characters");
23         validatePassword(password);
24         String id = UUID.randomUUID().toString();
25         byte[] salt = bytes(SALT_BYTES);
26         byte[] hash = hash(password, salt);
27         try (Connection connection = open(); PreparedStatement statement = connection.prepareStatement(
28             "insert into users(id,email,display_name,password_hash,password_salt,disabled,created_at)
29 > values(?,?,?,?,0,?)") {
30             statement.setString(1, id); statement.setString(2, normalizedEmail); statement.setString(3,
31 > displayName.trim());
32             statement.setBytes(4, hash); statement.setBytes(5, salt); statement.setString(6,
33 > Instant.now().toString()); statement.executeUpdate();
34             try (PreparedStatement role = connection.prepareStatement("insert into user_roles(user_id,role)
35 > values(?, 'STUDENT')") { role.setString(1, id); role.executeUpdate(); }
36             } catch (SQLException error) { if (error.getMessage().contains("UNIQUE")) throw new
37 > SecurityException("duplicate account"); throw database(error); }
38             return issue(user(id));
39         }
40     void ensureBootstrapAdmin(String email, char[] password) {
41         String normalizedEmail = validateEmail(email);
42         try (Connection connection = open();
43             PreparedStatement find = connection.prepareStatement("select id from users where email=?") {
44             find.setString(1, normalizedEmail);
45             try (ResultSet existing = find.executeQuery()) {
46                 String userId;
47                 if (existing.next()) {
48                     userId = existing.getString(1);
49                 } else {
50                     userId = registerData(normalizedEmail, "System Administrator", password).user().id();
```

```
01         }
02         try (PreparedStatement role = connection.prepareStatement(
03             "insert or ignore into user_roles(user_id,role) values(?, 'ADMIN')")) {
04             role.setString(1, userId);
05             role.executeUpdate();
06         }
07     }
08 } catch (SQLException error) {
09     throw database(error);
10 } finally {
11     java.util.Arrays.fill(password, '\0');
12 }
13 }
14 SessionData loginData(String email, char[] password) {
15     String normalizedEmail = validateEmail(email); validatePassword(password);
16     String userId;
17     try (Connection connection = open(); PreparedStatement statement = connection.prepareStatement(
18         "select id,password_hash,password_salt,disabled from users where email=?")) {
19         statement.setString(1, normalizedEmail);
20         try (ResultSet result = statement.executeQuery()) {
21             if (!result.next() || result.getInt("disabled") != 0 ||
22 > !constantTimeEquals(result.getBytes("password_hash"), hash(password, result.getBytes("password_salt")))) throw new
23 > SecurityException("invalid credentials");
24             userId = result.getString("id");
25         }
26     } catch (SQLException error) { throw database(error); }
27     return issue(user(userId));
28 }
29 @Override public Classroom create(AuthenticatedUser actor, String name) {
30     if (!(actor.hasRole(UserRole.TEACHER) || actor.hasRole(UserRole.ADMIN))) throw new
31 > SecurityException("teacher role required");
32     if (name == null || name.isBlank() || name.length() > 100) throw new IllegalArgumentException("name must
33 > be 1 to 100 characters");
34     String id = UUID.randomUUID().toString(); Instant now = Instant.now();
35     try (Connection connection = open(); PreparedStatement classroom = connection.prepareStatement("insert
36 > into classrooms(id,name,created_at) values(?,?,?)"); PreparedStatement member = connection.prepareStatement("insert
37 > into classroom_members(classroom_id,user_id,role) values(?,?,?)")) {
38         connection.setAutoCommit(false); classroom.setString(1,id); classroom.setString(2,name.trim());
39 > classroom.setString(3,now.toString()); classroom.executeUpdate();
40         member.setString(1,id); member.setString(2,actor.id()); member.setString(3,UserRole.TEACHER.name());
41 > member.executeUpdate(); connection.commit();
42     } catch (SQLException error) { throw database(error); }
43     return classroom(id);
44 }
45 @Override public Classroom addMember(AuthenticatedUser actor, String classroomId, String userId, UserRole
46 > role) {
47     if (role == null || role == UserRole.ADMIN) throw new IllegalArgumentException("Only TEACHER or STUDENT
48 > can join a classroom");
49     requireTeacher(actor, classroomId);
50     try (Connection connection = open();
```

```
01         PreparedStatement statement = connection.prepareStatement(
02             "insert into classroom_members(classroom_id,user_id,role) values(?,?,?) "
03             + "on conflict(classroom_id,user_id) do update set role=excluded.role");
04         PreparedStatement promoteTeacher = connection.prepareStatement(
05             "insert or ignore into user_roles(user_id,role) values(?, 'TEACHER')") {
06         connection.setAutoCommit(false);
07         statement.setString(1, classroomId);
08         statement.setString(2, userId);
09         statement.setString(3, role.name());
10         statement.executeUpdate();
11         if (role == UserRole.TEACHER) {
12             promoteTeacher.setString(1, userId);
13             promoteTeacher.executeUpdate();
14         }
15         connection.commit();
16     } catch (SQLException error) { throw database(error); }
17     return classroom(classroomId);
18 }
19 @Override public List<Classroom> listVisibleTo(AuthenticatedUser actor) {
20     List<Classroom> classrooms = new ArrayList<>();
21     String sql = actor.hasRole(UserRole.ADMIN) ? "select id from classrooms order by created_at desc" :
22 > "select c.id from classrooms c join classroom_members m on m.classroom_id=c.id where m.user_id=? order by c.created_at
23 > desc";
24     try (Connection connection = open(); PreparedStatement statement = connection.prepareStatement(sql)) {
25         if (!actor.hasRole(UserRole.ADMIN)) statement.setString(1,actor.id());
26         try (ResultSet result = statement.executeQuery()) { while (result.next())
27 > classrooms.add(classroom(result.getString(1))); }
28     } catch (SQLException error) { throw database(error); }
29     return List.copyOf(classrooms);
30 }
31 int upload(AuthenticatedUser actor, List<CloudSyncItem> items) {
32     if (items.size() > 500) throw new IllegalArgumentException("A sync batch may contain at most 500 items");
33     try (Connection connection = open(); PreparedStatement statement = connection.prepareStatement(
34         "insert into sync_events(user_id,event_id,event_type,payload_json,occurred_at) values(?,?,?,?) "
35         + "on conflict(user_id,event_id) do update set
36 > event_type=excluded.event_type,payload_json=excluded.payload_json,occurred_at=excluded.occurred_at")) {
37         connection.setAutoCommit(false);
38         for (CloudSyncItem item : items) {
39             statement.setString(1, actor.id()); statement.setString(2, item.id()); statement.setString(3,
40 > item.type());
41             statement.setString(4, item.payloadJson()); statement.setString(5, item.occurredAt().toString());
42 > statement.addBatch();
43         }
44         statement.executeBatch(); connection.commit(); return items.size();
45     } catch (SQLException error) { throw database(error); }
46 }
47 List<CloudSyncItem> download(AuthenticatedUser actor, long afterVersion) {
48     if (afterVersion < 0) throw new IllegalArgumentException("afterVersion must not be negative");
49     List<CloudSyncItem> items = new ArrayList<>();
50     try (Connection connection = open(); PreparedStatement statement = connection.prepareStatement(
```

```

01         "select event_id,event_type,payload_json,occurred_at,version from sync_events where user_id=? and
02         > version>? order by version limit 500")) {
03             statement.setString(1, actor.id()); statement.setLong(2, afterVersion);
04             try (ResultSet rows = statement.executeQuery()) {
05                 while (rows.next()) items.add(new CloudSyncItem(rows.getString(1), rows.getString(2),
06         > rows.getString(3), Instant.parse(rows.getString(4)), rows.getLong(5)));
07             }
08         } catch (SQLException error) { throw database(error); }
09         return List.copyOf(items);
10     }
11     ClassAssignment createAssignment(AuthenticatedUser actor,String classroomId,String exerciseId,String
12 > title){requireTeacher(actor,classroomId);if(exerciseId==null||exerciseId.isBlank()||title==null||title.isBlank())throw
13 > new IllegalArgumentException("exerciseId and title must not be blank");String id=UUID.randomUUID().toString();Instant
14 > now=Instant.now();try(Connection c=open());PreparedStatement s=c.prepareStatement("insert into
15 > class_assignments(id,classroom_id,exercise_id,title,created_at) values(?,?,?,?,?)"){s.setString(1,id);s.setString(2,c
16 > lassroomId);s.setString(3,exerciseId.trim());s.setString(4,title.trim());s.setString(5,now.toString());s.executeUpdate
17 > ();return new ClassAssignment(id,classroomId,exerciseId.trim(),title.trim(),now);}catch(SQLException e){throw
18 > database(e);}}
19     List<ClassAssignment> listAssignments(AuthenticatedUser actor,String
20 > classroomId){requireMember(actor,classroomId);List<ClassAssignment> result=new ArrayList<>();try(Connection
21 > c=open());PreparedStatement s=c.prepareStatement("select id,exercise_id,title,created_at from class_assignments where
22 > classroom_id=? order by created_at desc"){s.setString(1,classroomId);try(ResultSet
23 > r=s.executeQuery()){while(r.next())result.add(new
24 > ClassAssignment(r.getString(1),classroomId,r.getString(2),r.getString(3),Instant.parse(r.getString(4))));}return
25 > List.copyOf(result);}catch(SQLException e){throw database(e);}}
26     private AuthenticatedUser authenticateData(String token) {
27         try (Connection connection = open(); PreparedStatement statement = connection.prepareStatement("select
28 > user_id from access_tokens where token_hash=? and expires_at=? and revoked_at is null")) {
29             statement.setBytes(1, tokenHash(token)); statement.setString(2, Instant.now().toString());
30             try (ResultSet result = statement.executeQuery()) { if (!result.next()) throw new
31 > SecurityException("invalid token"); return user(result.getString(1)); }
32         } catch (SQLException error) { throw database(error); }
33     }
34     private void revoke(String token) { try (Connection c=open(); PreparedStatement s=c.prepareStatement("update
35 > access_tokens set revoked_at=? where token_hash=?") { s.setString(1,Instant.now().toString());
36 > s.setBytes(2,tokenHash(token)); if(s.executeUpdate()==0) throw new SecurityException("invalid token"); }
37 > catch(SQLException e){throw database(e);} }
38     private SessionData issue(AuthenticatedUser user) { String
39 > token=Base64.getUrlEncoder().withoutPadding().encodeToString(bytes(TOKEN_BYTES)); Instant expiry=Instant.now().plus(8,
40 > ChronoUnit.HOURS); try(Connection c=open(); PreparedStatement s=c.prepareStatement("insert into
41 > access_tokens(token_hash,user_id,expires_at,created_at) values(?,?,?,?,?)"){s.setBytes(1,tokenHash(token));s.setString(
42 > 2,user.id());s.setString(3,expiry.toString());s.setString(4,Instant.now().toString());s.executeUpdate();}catch(SQLExce
43 > ption e){throw database(e);} return new SessionData(token,expiry,user); }
44     private AuthenticatedUser user(String id) { try(Connection c=open(); PreparedStatement
45 > s=c.prepareStatement("select id,email,display_name from users where id=? and
46 > disabled=0")){s.setString(1,id);try(ResultSet r=s.executeQuery()){if(!r.next())throw new SecurityException("unknown
47 > user");Set<UserRole> roles=new java.util.HashSet<>();try(PreparedStatement rs=c.prepareStatement("select role from
48 > user_roles where user_id=?")){rs.setString(1,id);try(ResultSet
49 > rr=rs.executeQuery()){while(rr.next())roles.add(UserRole.valueOf(rr.getString(1)));}}return new
50 > AuthenticatedUser(r.getString(1),r.getString(2),r.getString(3),roles);}catch(SQLException e){throw database(e);} }

```

```
01     private String userIdByEmail(String email){String normalized=validateEmail(email);try(Connection
02 > c=open();PreparedStatement s=c.prepareStatement("select id from users where email=? and
03 > disabled=0")){s.setString(1,normalized);try(ResultSet r=s.executeQuery()){if(!r.next())throw new
04 > IllegalArgumentException("User email was not found");return r.getString(1);}}catch(SQLException e){throw
05 > database(e);}}
06     private Classroom classroom(String id) {
07         try (Connection connection = open();
08             PreparedStatement classroomStatement = connection.prepareStatement(
09                 "select name, created_at from classrooms where id=?") {
10             classroomStatement.setString(1, id);
11             try (ResultSet classroom = classroomStatement.executeQuery()) {
12                 if (!classroom.next()) {
13                     throw new IllegalArgumentException("Classroom not found");
14                 }
15                 List<Member> members = new ArrayList<>();
16                 try (PreparedStatement memberStatement = connection.prepareStatement(
17                     "select user_id, role from classroom_members where classroom_id=?") {
18                     memberStatement.setString(1, id);
19                     try (ResultSet memberRows = memberStatement.executeQuery()) {
20                         while (memberRows.next()) {
21                             members.add(new Member(memberRows.getString(1),
22 > UserRole.valueOf(memberRows.getString(2))));
23                         }
24                     }
25                 }
26                 return new Classroom(id, classroom.getString(1), Instant.parse(classroom.getString(2)), members);
27             }
28         } catch (SQLException error) {
29             throw database(error);
30         }
31     }
32     private void requireTeacher(AuthenticatedUser actor,String
33 > classId){if(actor.hasRole(UserRole.ADMIN))return;try(Connection c=open();PreparedStatement
34 > s=c.prepareStatement("select 1 from classroom_members where classroom_id=? and user_id=? and
35 > role='TEACHER'")){s.setString(1,classId);s.setString(2,actor.id());try(ResultSet
36 > r=s.executeQuery()){if(!r.next())throw new SecurityException("not classroom teacher");}}catch(SQLException e){throw
37 > database(e);}}
38     private void requireMember(AuthenticatedUser actor,String
39 > classId){if(actor.hasRole(UserRole.ADMIN))return;try(Connection c=open();PreparedStatement
40 > s=c.prepareStatement("select 1 from classroom_members where classroom_id=? and
41 > user_id=?")){s.setString(1,classId);s.setString(2,actor.id());try(ResultSet r=s.executeQuery()){if(!r.next())throw new
42 > SecurityException("not classroom member");}}catch(SQLException e){throw database(e);}}
43     private Connection open() throws SQLException { return DriverManager.getConnection("jdbc:sqlite:"+database); }
44     private void initialize() throws SQLException { try(Connection c=open(); Statement
45 > s=c.createStatement()){s.executeUpdate("pragma foreign_keys=on");s.executeUpdate("create table if not exists users(id
46 > text primary key,email text not null unique,display_name text not null,password_hash blob not null,password_salt blob
47 > not null,disabled integer not null default 0,created_at text not null");s.executeUpdate("create table if not exists
48 > user_roles(user_id text not null references users(id),role text not null check(role in
49 > ('ADMIN','TEACHER','STUDENT')),primary key(user_id,role))");s.executeUpdate("create table if not exists
50 > access_tokens(token_hash blob primary key,user_id text not null references users(id),expires_at text not
```

```

01 > null,created_at text not null,revoked_at text));s.executeUpdate("create table if not exists classrooms(id text
02 > primary key,name text not null,created_at text not null));s.executeUpdate("create table if not exists
03 > classroom_members(classroom_id text not null references classrooms(id),user_id text not null references users(id),role
04 > text not null check(role in ('TEACHER','STUDENT')),primary key(classroom_id,user_id));s.executeUpdate("create table
05 > if not exists class_assignments(id text primary key,classroom_id text not null references classrooms(id),exercise_id
06 > text not null,title text not null,created_at text not null));s.executeUpdate("create table if not exists
07 > sync_events(version integer primary key autoincrement,user_id text not null references users(id),event_id text not
08 > null,event_type text not null,payload_json text not null,occurred_at text not null,unique(user_id,event_id));} }
09 private byte[] bytes(int count){byte[] value=new byte[count];random.nextBytes(value);return value;} private
10 > byte[] hash(char[] password,byte[] salt){try{KeySpec spec=new
11 > PBEKeySpec(password,salt,PBKDF2_ITERATIONS,HASH_BITS);return
12 > SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256").generateSecret(spec).getEncoded();}catch(GeneralSecurityException
13 > e){throw new IllegalStateException("Password hashing unavailable",e);}} private byte[] tokenHash(String
14 > token){try{return java.security.MessageDigest.getInstance("SHA-256").digest(token.getBytes(StandardCharsets.UTF_8));}c
15 > atch(GeneralSecurityException e){throw new IllegalStateException(e);}} private static boolean
16 > constantTimeEquals(byte[] a,byte[] b){return java.security.MessageDigest.isEqual(a,b);} private static String
17 > validateEmail(String e){if(e==null||!e.matches("^[^\\s@]+@[^\\s@]+\\.([^\\s@]+)$")||e.length()>254)throw new
18 > IllegalArgumentException("email must be valid");return e.trim().toLowerCase(Locale.ROOT);} private static void
19 > validatePassword(char[] p){if(p==null||p.length<12||p.length>128)throw new IllegalArgumentException("password must
20 > contain 12 to 128 characters");} private static IllegalStateException database(SQLException e){return new
21 > IllegalStateException("Cloud database operation failed",e);} private static CloudAuthenticationService.Session
22 > toSession(SessionData s){return new CloudAuthenticationService.Session(s.token(),s.expiresAt(),s.user());}
23 }
24 private record SessionData(String token, Instant expiresAt, AuthenticatedUser user) { }
25 }
26 // FILE: src\\main\\java\\com\\sqlteacher\\StageOneVerificationApp.java
27 package com.sqlteacher;
28 import com.sqlteacher.application.ai.AiStatus;
29 import com.sqlteacher.application.ai.AiStatusService;
30 import com.sqlteacher.application.ai.AiModelSelection;
31 import com.sqlteacher.application.ai.AiModelSelectionService;
32 import com.sqlteacher.application.database.DatabaseInitializationResult;
33 import com.sqlteacher.application.database.DatabaseInitializationService;
34 import com.sqlteacher.infrastructure.spring.SqlTeacherApplicationConfig;
35 import org.springframework.context.annotation.AnnotationConfigApplicationContext;
36 public final class StageOneVerificationApp {
37 private StageOneVerificationApp() {
38 }
39 public static void main(String[] args) {
40 try (AnnotationConfigApplicationContext context =
41 new AnnotationConfigApplicationContext(SqlTeacherApplicationConfig.class)) {
42 DatabaseInitializationResult databaseResult =
43 context.getBean(DatabaseInitializationService.class).initialize();
44 AiStatus aiStatus = context.getBean(AiStatusService.class).checkStatus();
45 AiModelSelection modelSelection =
46 context.getBean(AiModelSelectionService.class).refresh();
47 System.out.println("SQLTeacher stage 1 application verification");
48 System.out.println("[PASS] Spring DI - application context started");
49 System.out.println("[PASS] SQLite app database - " + databaseResult.appDatabasePath());
50 System.out.println("[PASS] SQLite demo database - " + databaseResult.demoDatabasePath());

```

```
01         System.out.printf(
02             "[%s] Ollama status - %s\n",
03             aiStatus.available() ? "PASS" : "WARNING",
04             aiStatus.message()
05         );
06         System.out.printf(
07             "[%s] Ollama models - installed=%s, selected=%s\n",
08             modelSelection.hasSelection() ? "PASS" : "WARNING",
09             modelSelection.installedModels(),
10             modelSelection.hasSelection() ? modelSelection.selectedModel() : "none"
11         );
12     }
13 }
14 }
15 // FILE: src\main\java\com\sqlteacher\TechnologyVerificationApp.java
16 package com.sqlteacher;
17 import com.sqlteacher.infrastructure.ai.OllamaHealthClient;
18 import com.sqlteacher.infrastructure.database.JdbcTechnologyVerifier;
19 import com.sqlteacher.infrastructure.environment.JavaFxEnvironmentVerifier;
20 import com.sqlteacher.infrastructure.environment.RuntimeEnvironment;
21 import com.sqlteacher.infrastructure.environment.VerificationItem;
22 import java.net.URI;
23 import java.time.Duration;
24 import java.util.ArrayList;
25 import java.util.List;
26 public final class TechnologyVerificationApp {
27     private TechnologyVerificationApp() {
28     }
29     public static void main(String[] args) {
30         List<VerificationItem> items = new ArrayList<>();
31         RuntimeEnvironment environment = RuntimeEnvironment.detect();
32         JdbcTechnologyVerifier jdbcVerifier = new JdbcTechnologyVerifier();
33         JavaFxEnvironmentVerifier javaFxVerifier = new JavaFxEnvironmentVerifier();
34         OllamaHealthClient ollamaHealthClient = new OllamaHealthClient(
35             URI.create("http://localhost:11434/api/tags"),
36             Duration.ofSeconds(2)
37         );
38         items.add(environment.javaVersionItem());
39         items.add(environment.osItem());
40         items.add(javaFxVerifier.verifyRuntime());
41         items.add(javaFxVerifier.verifyGraphicsEnvironment());
42         items.add(jdbcVerifier.verifySqliteInMemoryQuery());
43         items.add(jdbcVerifier.verifyMySQLDriverAvailable());
44         items.add(jdbcVerifier.verifyMariaDbDriverAvailable());
45         items.add(ollamaHealthClient.checkHealth());
46         System.out.println("SQLTeacher stage 0 technology verification");
47         for (VerificationItem item : items) {
48             System.out.printf("[%s] %s - %s\n", item.status(), item.name(), item.detail());
49         }
50     }
51 }
```